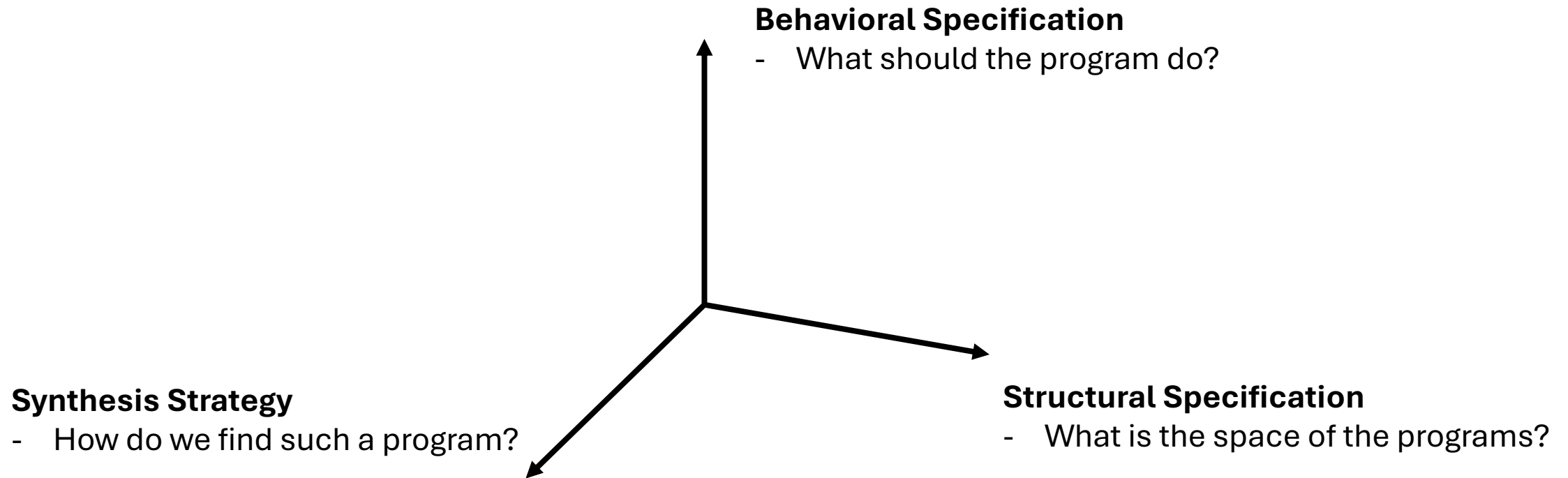


Machine Programming

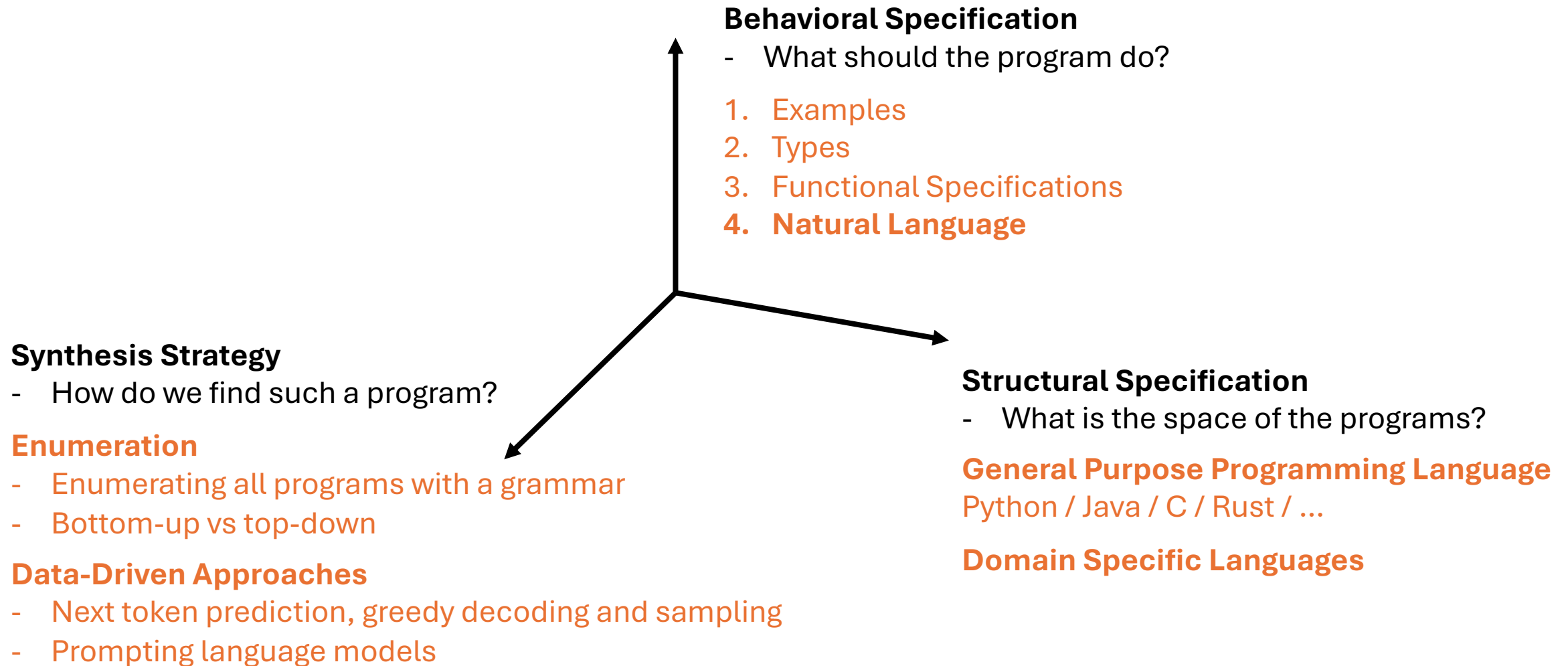
Lecture 7 – Iterative Refinement and Agentic Framework

Ziyang Li

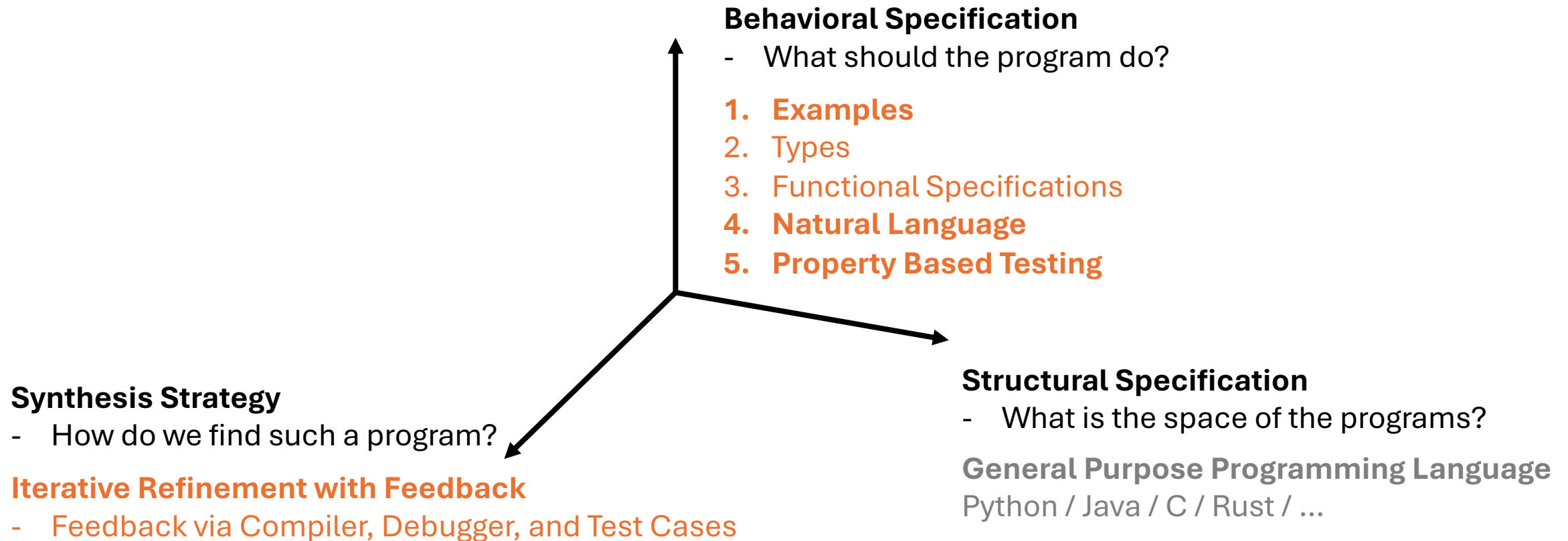
Dimensions in Program Synthesis



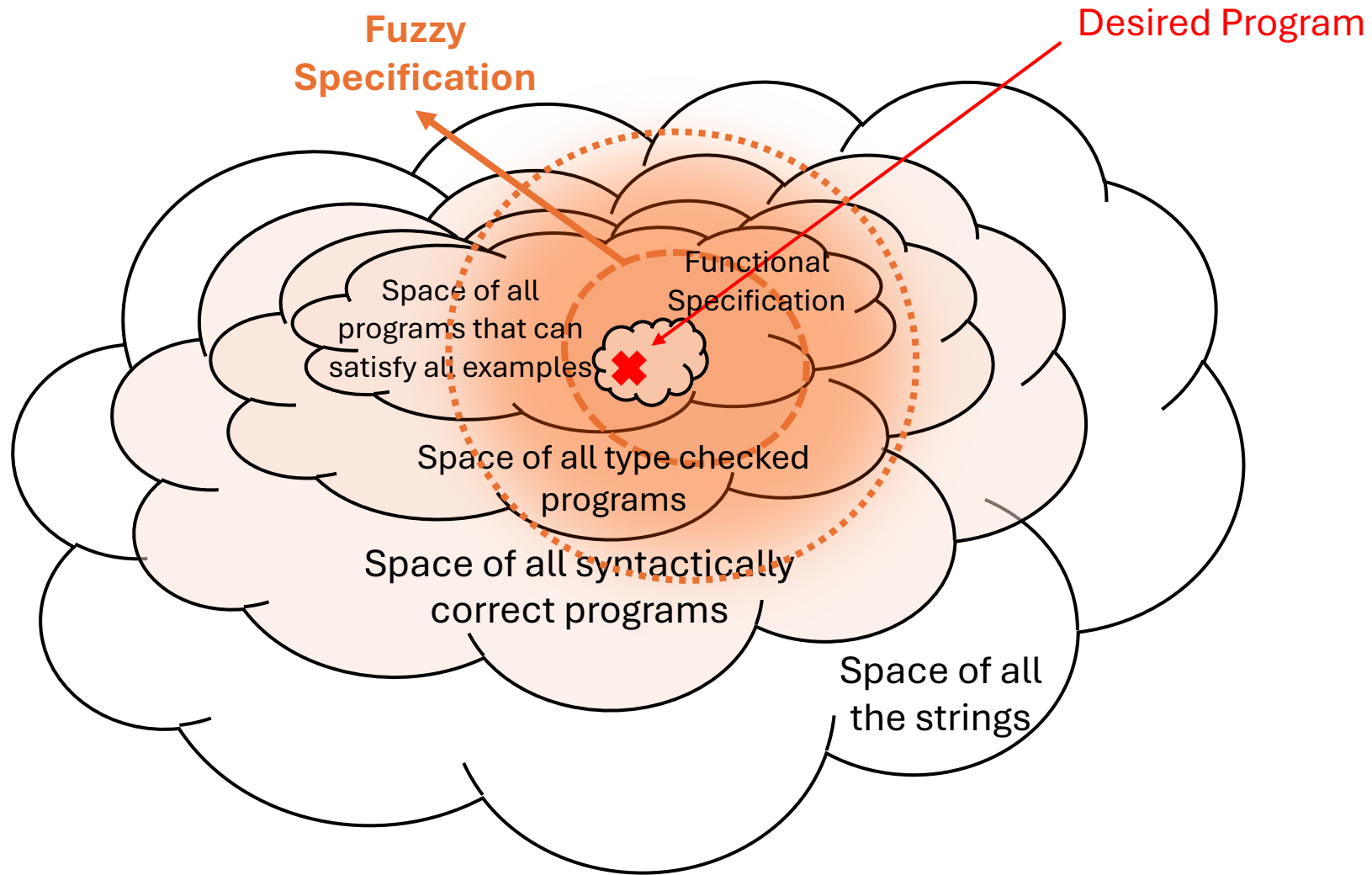
The Course So Far



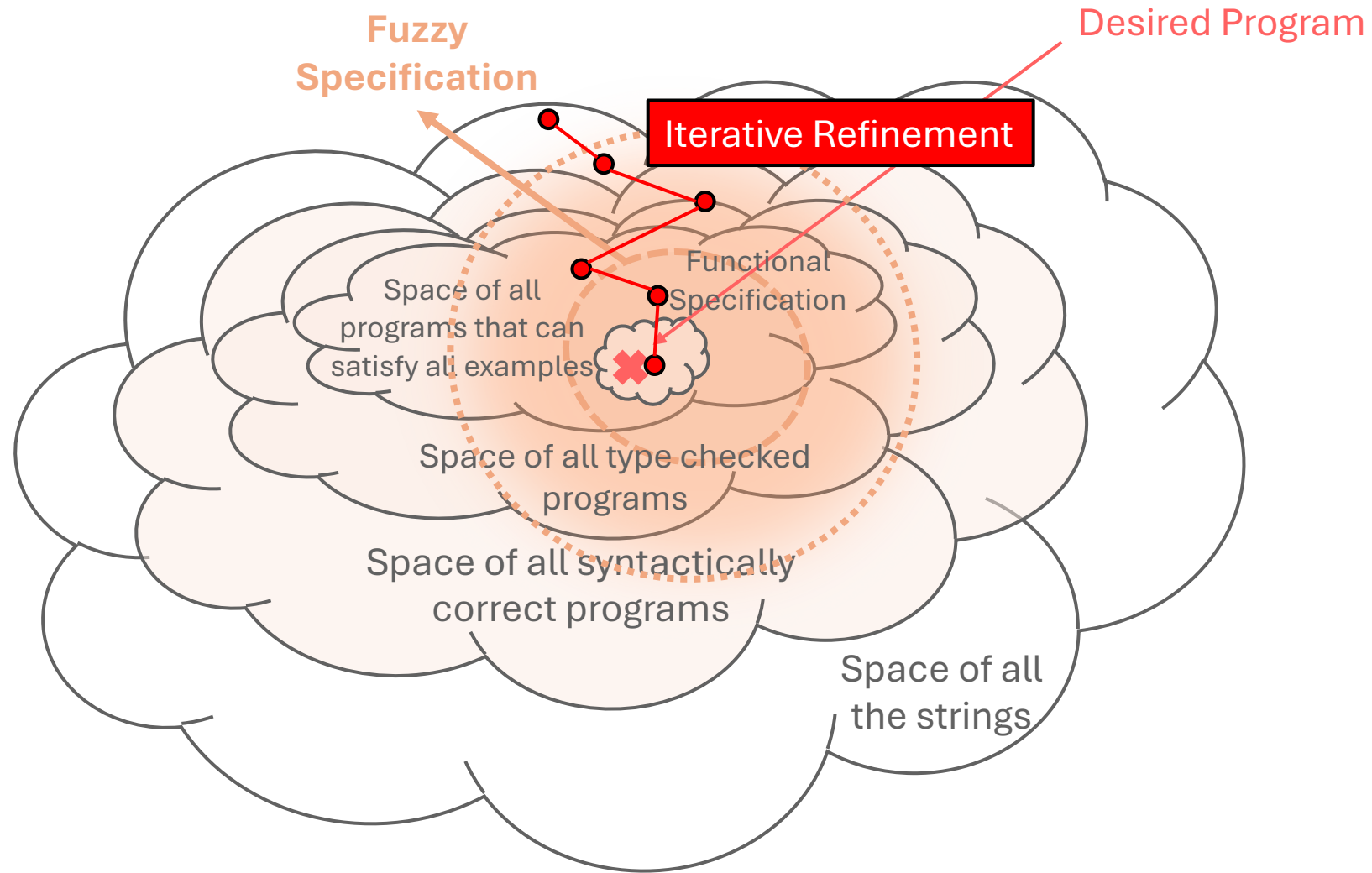
Today



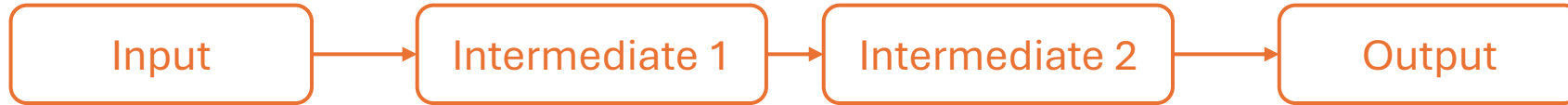
High Level Picture



High Level Picture



From Multi-Step Prompting to Iterative Prompting



Unifying Inference-Time Planning Language Generation

Prabhu Prakash Kagitha  Bo Sun  Ishan Desai  Andrew Zhu 
Cassie Huang  Manling Li  Ziyang Li  Li Zhang 
 Drexel University  University of Pennsylvania
 Northwestern University  Johns Hopkins University
{prabhuprakash.k, harry.zhang}@drexel.edu

Unifying Inference-Time Planning Language Generation

Prabhu Prakash Kagitha



Bo Sun



Ishan Desai



Andrew Zhu



Cassidy

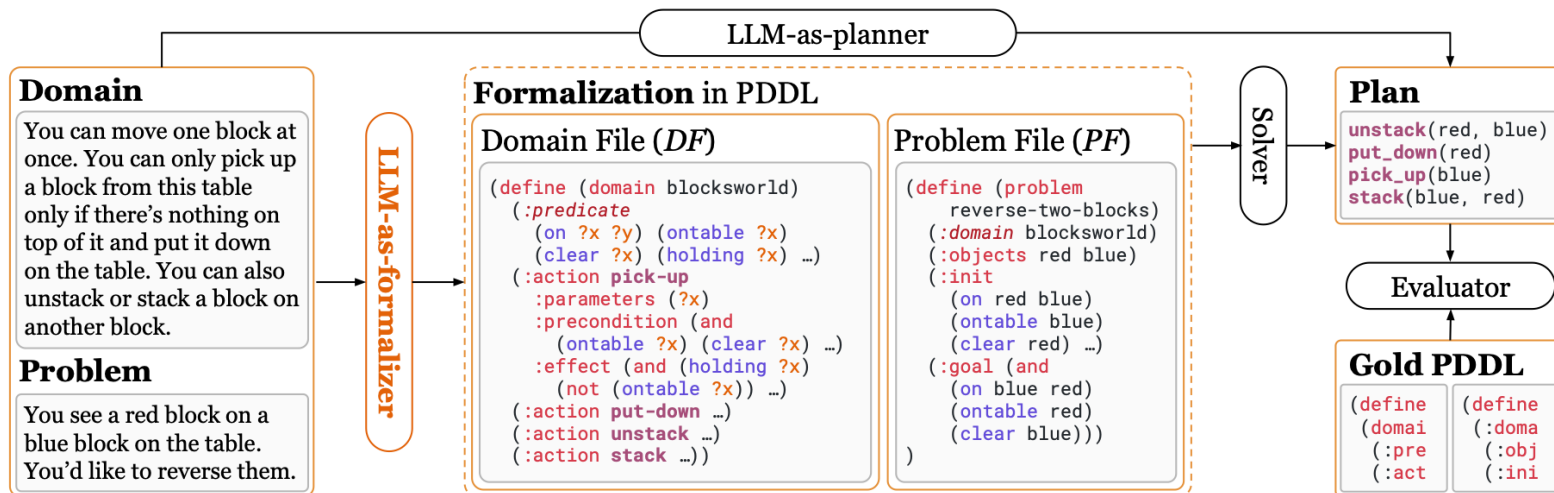


Figure 1: An illustration of using LLM as a planner or a formalizer in classical planning. While *LLM-as-planner* generates a plan directly, *LLM-as-formalizer* formalizes a PDDL domain file and problem which evoke a solver to find a plan. The plan is evaluated against ground-truth PDDL that simulates the environment.

Unifyin

Prabl

Domain

You can move blocks once. You can only move a block from one table to another if there is space on the table. You can unstack or stack blocks on another block.

Problem

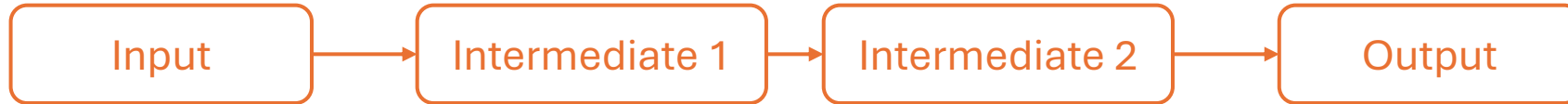
You see a room with a blue block and a red block. You'd like to move the blue block to the red block.

Figure 1: The LLM generates a plan to find a place for the blue block.

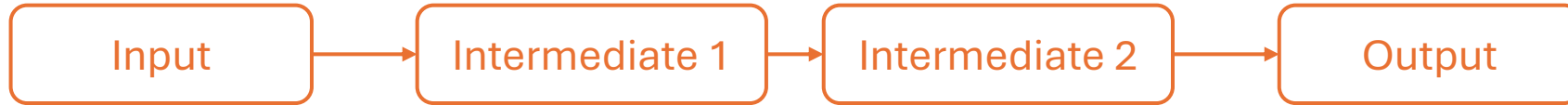
Lv.	Input	1st IR	2nd IR	3rd IR	Output	Description
0	I				$\rightarrow L$	LLM-as-planner (ϕ)
1	I	$\rightarrow$ PDDL			$\rightsquigarrow L$	<i>PDDL-base</i> : directly generating PDDL.
	I	$\rightarrow^{\text{GD}}$ PDDL			$\rightsquigarrow L$	<i>PDDL-grammar</i> [†] the above plus constraining LLM decoding with grammar.
2	I	$\rightarrow$ NL	$\rightarrow$ PDDL		$\rightsquigarrow L$	Generating an NL description of the actions and entity states before PDDL
	I	$\rightarrow$ PyPDDL	$\rightarrow \mid \rightsquigarrow$ PDDL		$\rightsquigarrow L$	[†] Transpiling a PyPDDL program to PDDL either with Py2PDDL ($\rightsquigarrow$) or by an LLM ($\rightarrow$).
	I	$\rightarrow$ PySim	$\rightarrow$ PDDL		$\rightsquigarrow L$	[†] Generating a Python-based simulator before translating to PDDL.
	I	$\rightarrow$ PDDL	$\rightarrow^{\text{FB?}}$ PDDL		$\rightsquigarrow L$	Refining the generated PDDL once (with or without the solver feedback).
3	I	$\rightarrow$ PyPDDL	$\rightarrow$ PDDL	$\rightarrow$ PDDL	$\rightsquigarrow L$	[†] Generating a PyPDDL wrapper before generating PDDL that receives one revision.
	I	$\rightarrow$ PyPDDL	$\rightarrow$ PyPDDL	$\rightarrow$ PDDL	$\rightsquigarrow L$	[†] Revising the PyPDDL representation once before generating PDDL.
	I	$\rightarrow$ NL	$\rightarrow$ PyPDDL	$\rightarrow$ PDDL	$\rightsquigarrow L$	[†] Marching through three IRs to emulate a loose-to-formal transpilation process.
	I	$\rightarrow$ PDDL	$\rightarrow^{\text{FB?}}$ PDDL	$\rightarrow^{\text{FB?}}$ PDDL	$\rightsquigarrow L$	Iteratively refining the PDDL program twice (with or without solver feedback).

Table 1: Pipelines that we evaluate in this work, ranging from level 0 to level 3. For each pipeline, we specify the IRs before reaching the final plan (L). Pipelines marked with [†] are newly introduced in this work. Straight arrows ($\rightarrow$) denote procedures involving LLMs, with superscripts indicating special variants—GD for grammar-guided decoding and FB for solver feedback. Wavy arrows ($\rightsquigarrow$) represent purely symbolic solving or compilation processes.

From Multi-Step Prompting to Iterative Prompting



From Multi-Step Prompting to Iterative Prompting

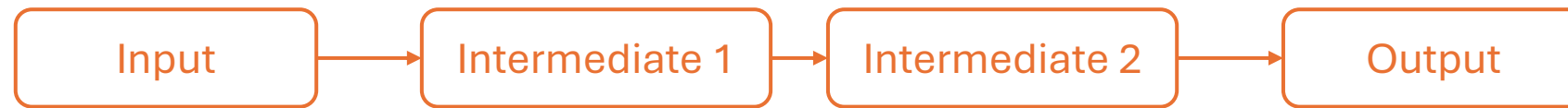


A known and fixed number of steps



A variable number of steps

From Multi-Step Prompting to Iterative Prompting



A known and fixed number of steps

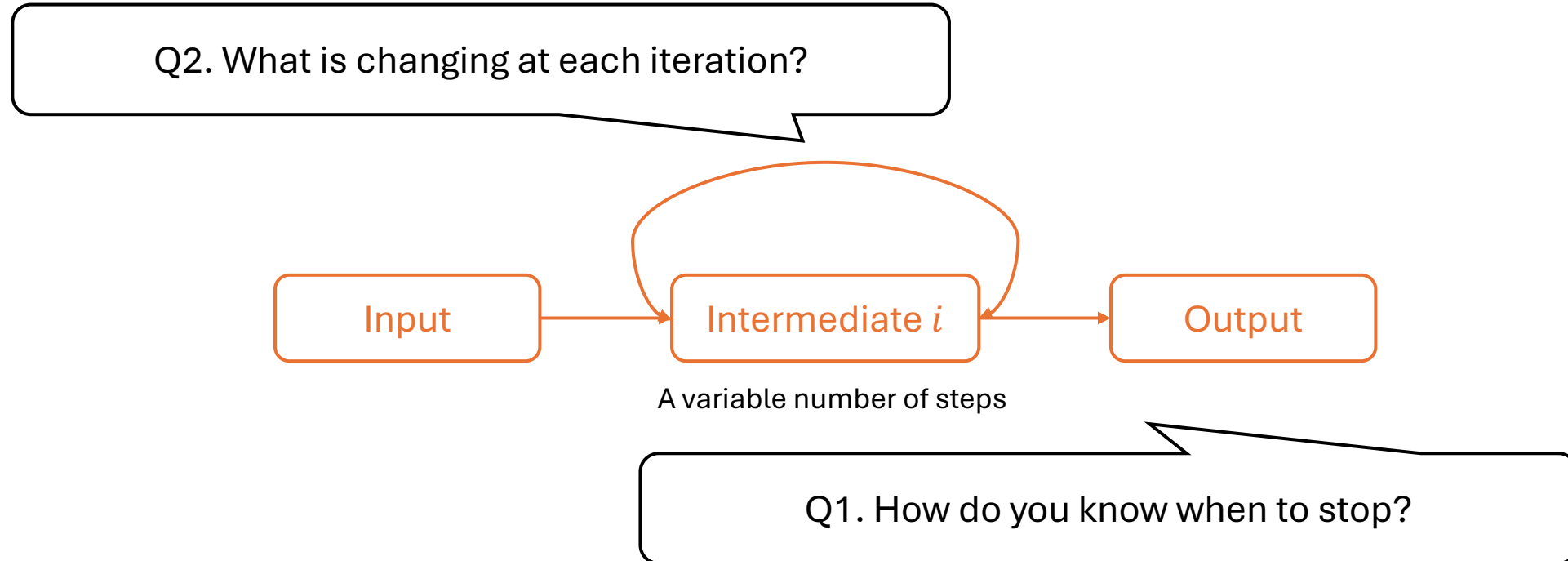
- Zero-shot (0)
- Few-shot (0)
- Chain-of-thought (1)
- Self-Reflection (1)

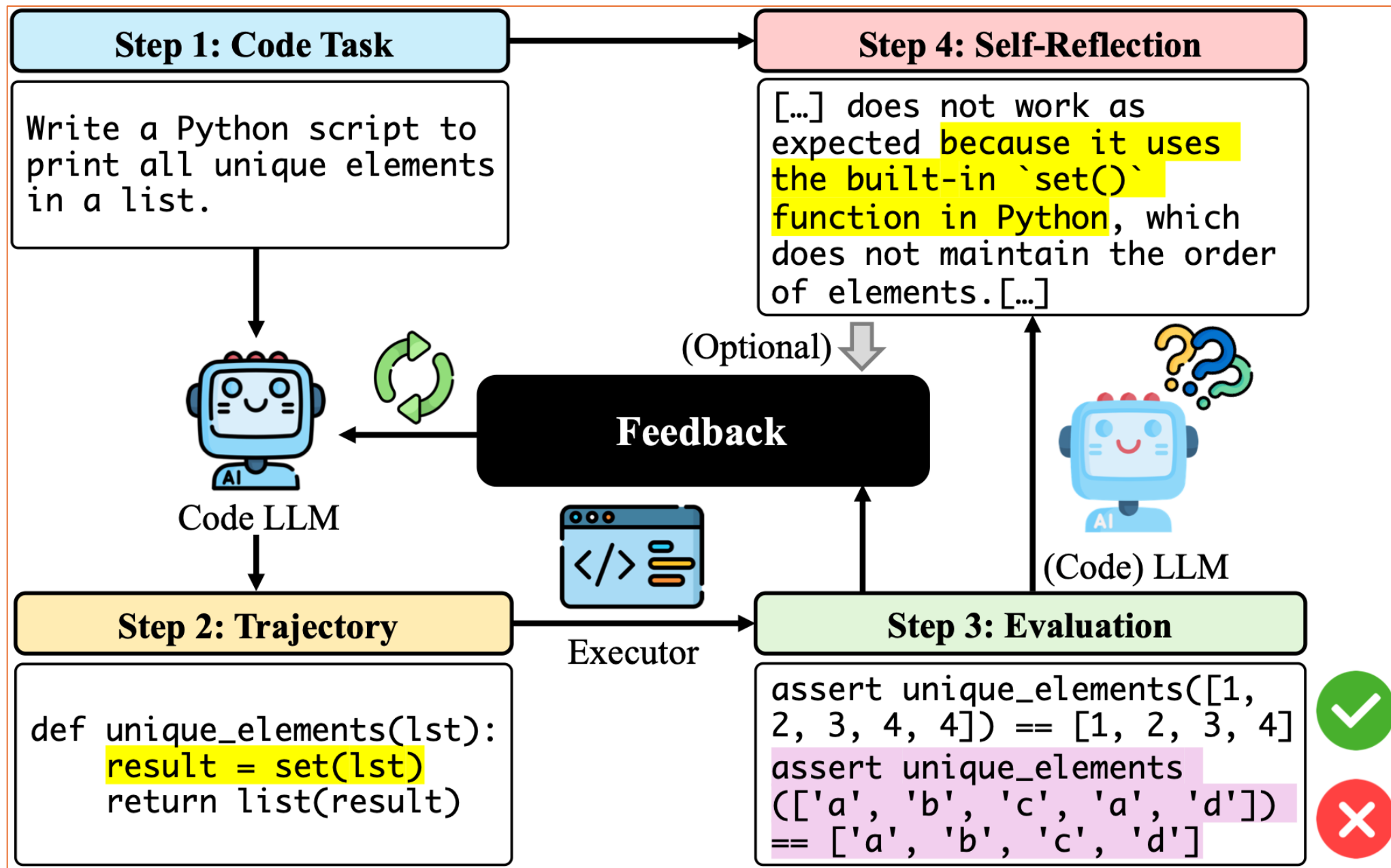


A variable number of steps

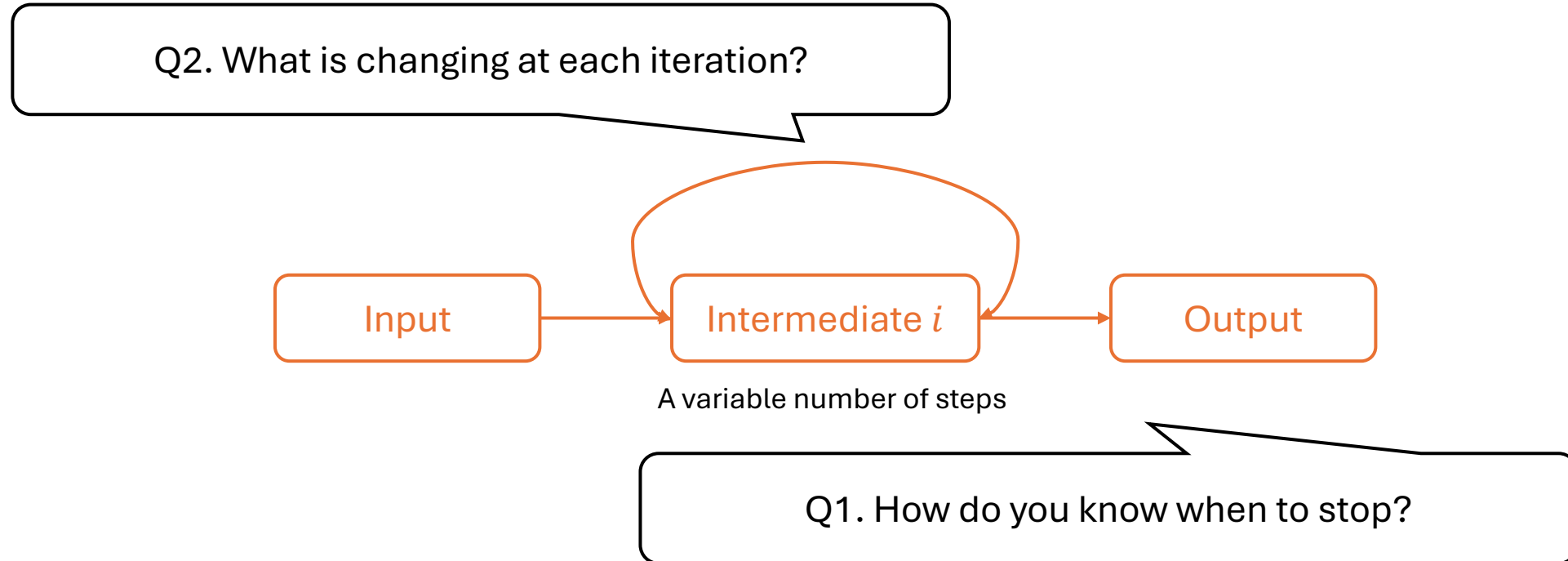
- Self-Debug
- Self-Repair
- FunSearch
- Evolutionary Search
- Generational Search

From Multi-Step Prompting to Iterative Prompting

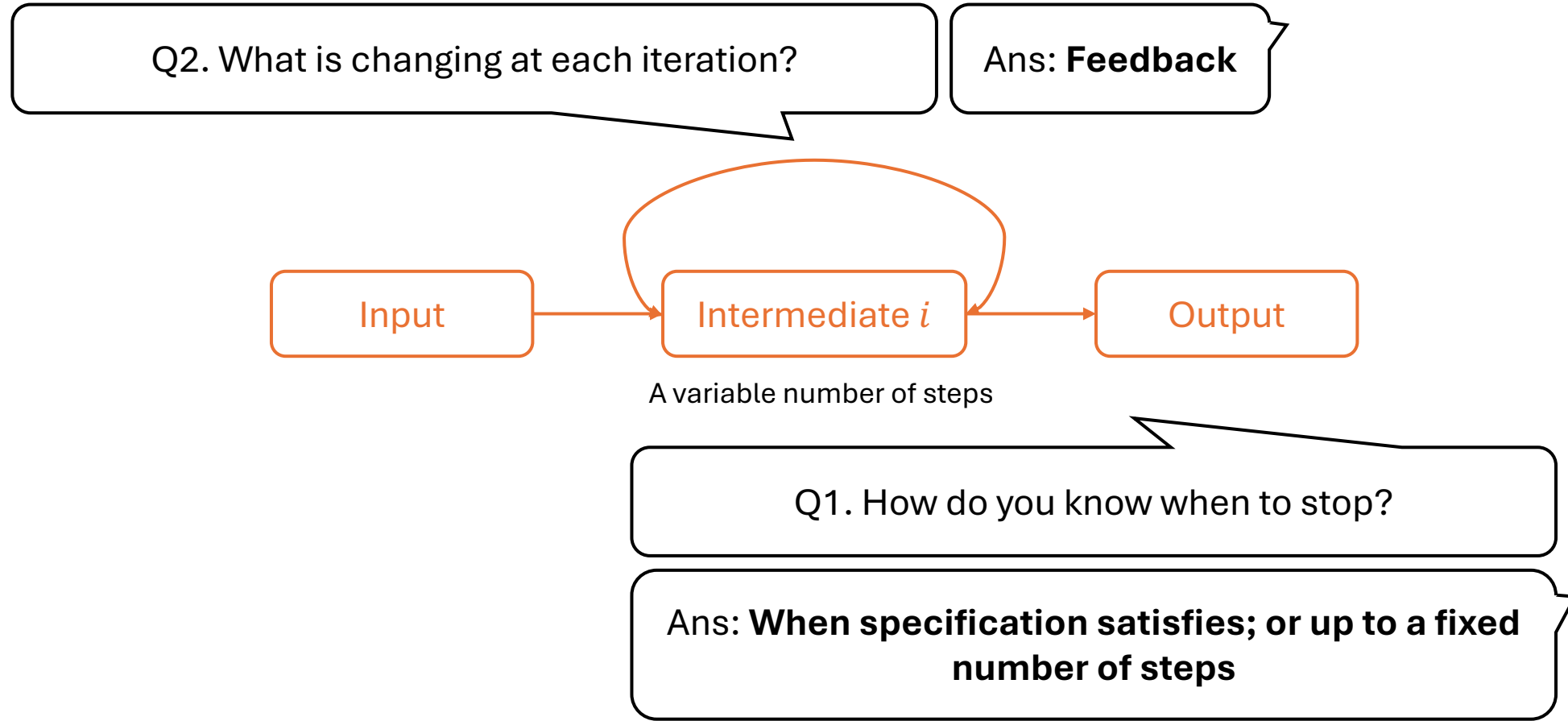




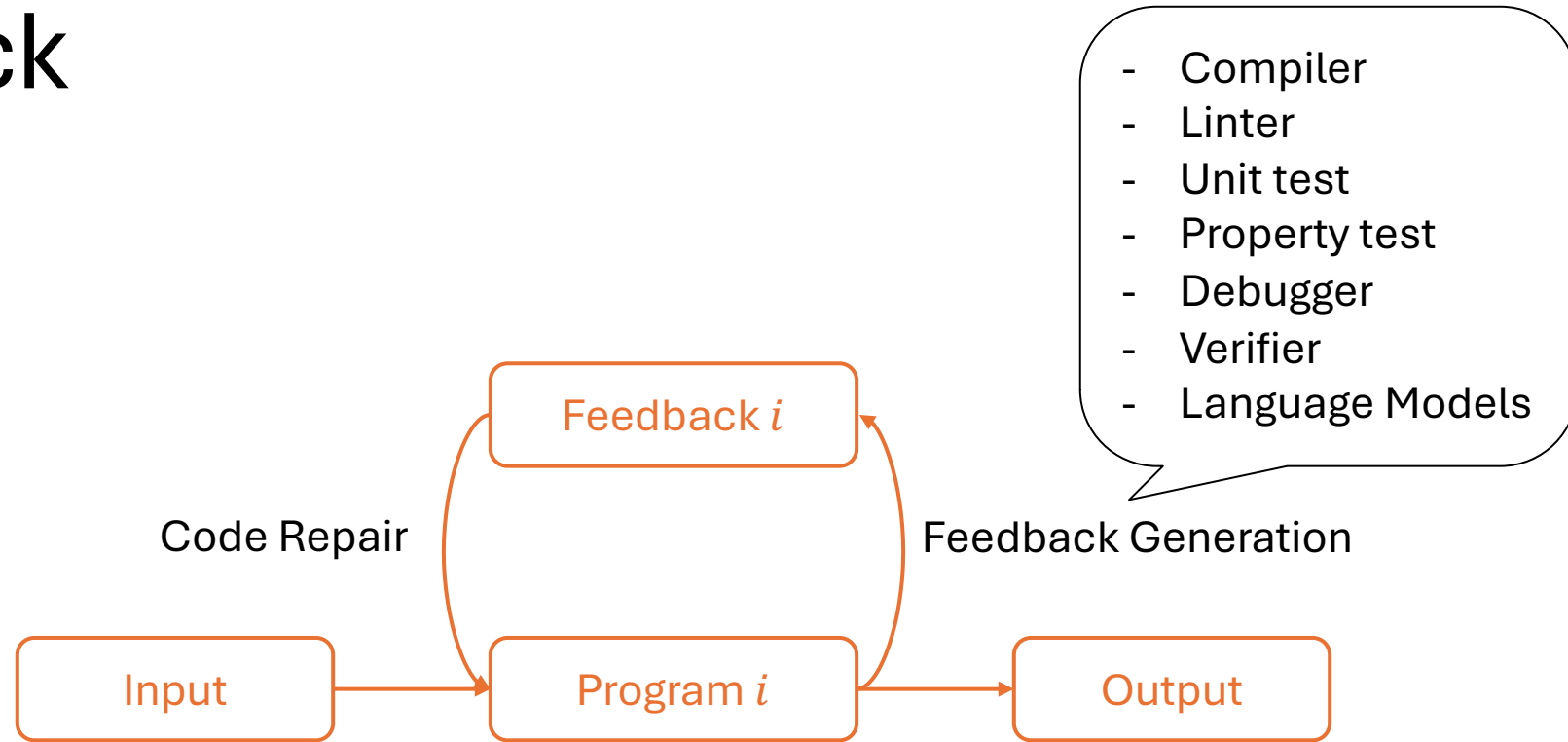
From Multi-Step Prompting to Iterative Prompting

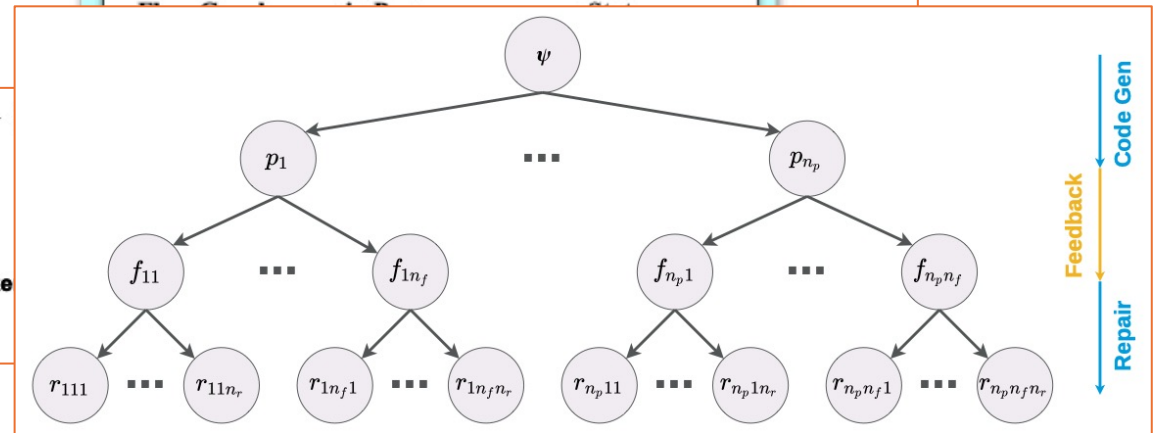
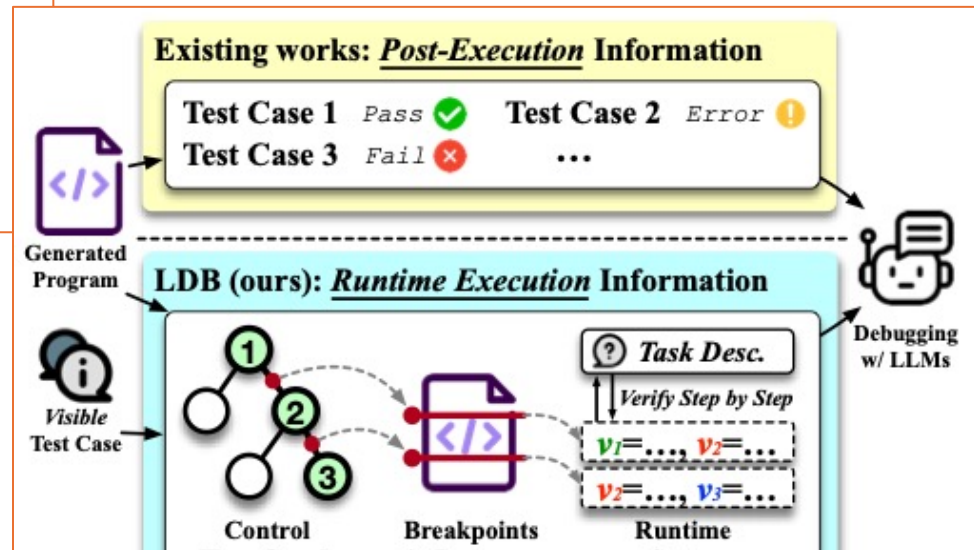
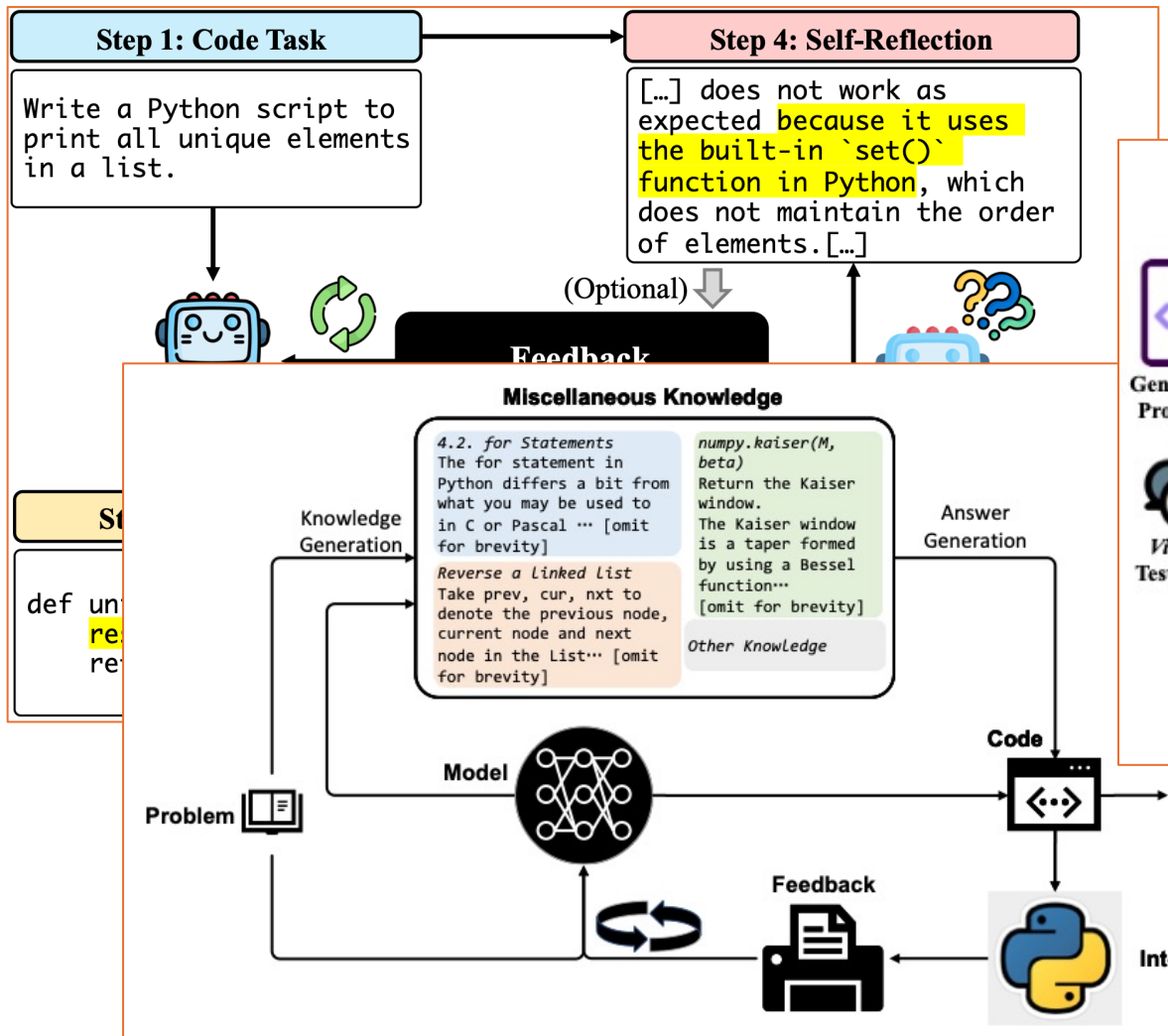


From Multi-Step Prompting to Iterative Prompting

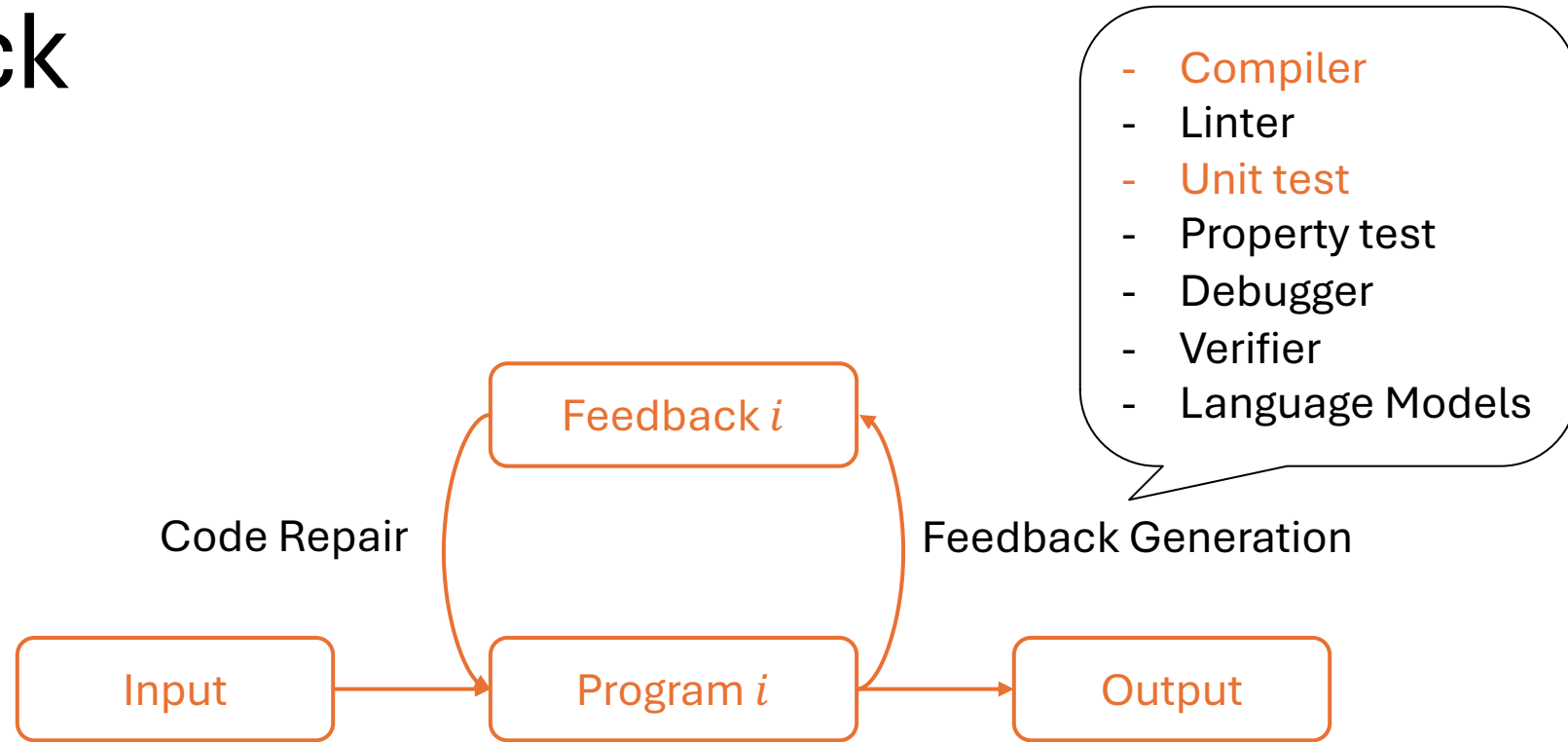


Feedback





Feedback



Compiler Feedback

- Classes of feedback
 - Syntax errors
 - Type checking errors
 - Language specific errors (type cast, memory, lifetime, etc.)
- Issues of too many errors
 - Verbose errors from lexer/parser/linter
 - Root error cascades and induce more errors
- Solutions
 - Heuristics: feed first few errors first!
 - Find root cause
 - Group errors by files

```
expression of type 'int' [-Wint-conversion]
30 | char *not_a_ptr = 42;           // error: invalid conversion
    | ^~~~~~
a.c:31:6: warning: incompatible pointer types initializing 'int **' with an expression of type 'int **'; remove & [-Wincompatible-pointer-types]
31 | int *ptr_to_self = &ptr_to_self; // error: invalid initializer
    | ^~~~~~
a.c:35:6: error: conflicting types for 'f'
35 | void f(double);                 // error: conflicting types for 'f'
    | ^~~~~~
a.c:34:6: note: previous declaration is here
34 | void f(int);
    | ^~~~~~
a.c:36:5: error: conflicting types for 'f'
36 | int f(int x) { return "nope"; } // error: returning wrong type (string to int)
    | ^~~~~~
a.c:34:6: note: previous declaration is here
34 | void f(int);
    | ^~~~~~
a.c:36:23: error: incompatible pointer to integer conversion returning 'char[5]' from a function with result type 'int' [-Wint-conversion]
36 | int f(int x) { return "nope"; } // error: returning wrong type (string to int)
    | ^~~~~~
a.c:38:5: warning: a function declaration without a prototype is deprecated in all versions of C and is treated as a zero-parameter prototype in C23, conflicting with a subsequent definition [-Wdeprecated-non-prototype]
38 | int g();                        // old-style declaration
    | ^~~~~~
a.c:39:5: note: conflicting prototype is here
39 | int g(int x) { return x; }      // error: conflicting types for 'g'
    | ^~~~~~
a.c:43:5: error: 'break' statement not in loop or switch statement
43 | break;                          // error: 'break' not in loop or switch
    | ^~~~~~
a.c:47:5: error: 'continue' statement not in loop statement
47 | continue;                       // error: 'continue' not in loop
    | ^~~~~~
a.c:51:5: error: 'case' statement not in switch statement
51 | case 3: printf("nope\n");       // error: 'case' label not within a switch statement
    | ^~~~~~
a.c:66:1: error: redefinition of label 'label'
66 | label:                          // error: duplicate label
    | ^~~~~~
a.c:64:1: note: previous definition is here
64 | label:
    | ^~~~~~
a.c:71:36: error: argument may not have 'void' type
71 | void nonsense_params(register void v) { // error: 'void' parameter variable
    | ^~~~~~
a.c:77:10: error: expected ';' at end of declaration list
77 | int x
    | ^~~~~~
fatal error: too many errors emitted, stopping now [-ferror-limit=]
4 warnings and 20 errors generated.
```

Linter Feedback

- What is a Linter
 - Programming errors and bugs
 - Stylistic inconsistencies
 - Suspicious constructs or “code smells”
 - Potential security vulnerabilities
- Catch
 - Usually very local
 - Many “false positives”
 - Most linter errors do not affect execution

```
liby99@hanabi ~/L/P/m/lecture-7-demos> npx eslint bad.js
/Users/liby99/Local/Projects/machine-programming/lecture-7-demos/bad.js
  4:1  error  'foo' is not defined
no-undef
  7:5  error  'neverChanges' is assigned a value but never used
no-unused-vars
 10:7  error  'greeting' is assigned a value but never used
no-unused-vars
 13:14 error  'foo' is not defined
no-undef
 13:20 error  'console' is not defined
no-undef
 16:10 error  'doStuff' is defined but never used
no-unused-vars
 17:3  error  'console' is not defined
no-undef
 18:17 error  Empty block statement
no-empty
 20:3  error  Unreachable code
no-unreachable
 20:3  error  'console' is not defined
no-undef
 24:21 error  Duplicate key 'a'
no-dupe-keys
 25:9  error  Do not access Object.prototype method 'hasOwnProperty' from
no-prototype-builtins
 26:3  error  'console' is not defined
no-undef
 30:10 error  'shadow' is defined but never used
no-unused-vars
 31:7  error  'x' is already defined
no-redeclare
 36:10 error  'parseStuff' is defined but never used
no-unused-vars
 41:7  error  'wrappedStr' is assigned a value but never used
no-unused-vars
 42:7  error  'wrappedNum' is assigned a value but never used
no-unused-vars
 45:1  error  Unexpected newline between object and [ of property access
no-unexpected-multiline
```

Linter Feedback

- What is a Linter
 - Programming errors and bugs
 - Stylistic inconsistencies
 - Suspicious constructs or “code smells”
 - Potential security vulnerabilities
- Catch
 - Usually very local
 - Many “false positives”
 - Most linter errors do not affect execution
 - Should be careful when presented to a language model

Help me write a javascript function `parity(x)` to find the parity of a given number.

Sure! In number theory / computer science, the **parity** of an integer usually means whether it has an **even** or **odd** number of **1** bits in its binary representation. Sometimes people just mean “is the number itself even or odd.” I’ll show you both:

```
JS parity.js > ...
1  function parity(x) {
2      let result = 0;
3      while (x > 0) {
4          result ^= (x & 1); // XOR with least significant bit
5          x >>= 1;           // shift right
6      }
7      return result; // 0 for even parity, 1 for odd parity
8  }
9
10 // Examples
11 console.log(parity(7)); // 1 → because 7 (111b) has three 1s
12 console.log(parity(12)); // 0 → because 12 (1100b) has two 1s
```

```
liby99@hanabi ~/L/P/m/lecture-7-demos> npx eslint parity.js
```

```
/Users/liby99/Local/Projects/machine-programming/lecture-7-demos/parity.js
11:1  error  'console' is not defined  no-undef
12:1  error  'console' is not defined  no-undef
```

```
* 2 problems (2 errors, 0 warnings)
```

```
liby99@hanabi ~/L/P/m/lecture-7-demos [1]> node parity.js
```

```
1
0
```

False positives

Unit Test Feedback

- Unit Tests
 - Encode basic input-output pairs
 - Usually provided along with the synthesis task as a part of behavioral specifications
- Encoding
 - As assertion
 - Using unit test libraries

```
parity_buggy.py > ...
1  def parity(x: int) -> int:
2      if x == 0:
3          return 1 # BUG
4
5      res = 0
6      while x:
7          res ^= (x & 1)
8          x >>= 1
9      return res
10
11 # ----- Tests (simple assertions) -----
12 assert parity(0) == 0, "parity(0) should be 0 (even), but
13 assert parity(1) == 1, "1 -> 1 (binary), odd -> 1"
14 assert parity(2) == 1, "2 -> 10 (binary), one '1' -> 1"
```

```
liby99@hanabi ~/L/P/m/lecture-7-demos> python3 parity_buggy.py
Traceback (most recent call last):
  File "/Users/liby99/Local/Projects/machine-programming/lecture-7-demos/parity_buggy.py",
    line 13, in <module>
      assert parity(0) == 0, "parity(0) should be 0 (even), but the function returns 1"
      ~~~~~
AssertionError: parity(0) should be 0 (even), but the function returns 1
```

Unit Test Feedback

- Unit Tests
 - Encode basic input-output pairs
 - Usually provided along with the synthesis task as a part of behavioral specifications
- Encoding
 - As assertion
 - Using **unit test libraries**

```
class TestParity(unittest.TestCase):  
    def test_zero_even_parity(self):  
        # Fails with the buggy implementation  
        self.assertEqual(parity(0), 0)
```

Python (using unittest)

```
#[test]  
fn zero_has_even_parity() {  
    // Fails with the buggy implementation  
    assert_eq!(parity(0), 0);  
}
```

Rust

```
spec :: Spec  
spec = do  
    describe "parity" $ do  
        it "returns 0 for zero (even parity)" $ do  
            parity 0 `shouldBe` 0
```

Haskell (using Test.Hspec)

```
TEST(Parity, ExamplesTable) {  
    struct Case { uint64_t x; int expected; };  
    Case cases[] = { {1, 1}, {2, 1}, {3, 0}, {7, 1} };  
    for (auto c : cases) {  
        EXPECT_EQ(parity(c.x), c.expected) << "x = " << c.x;  
    }  
}
```

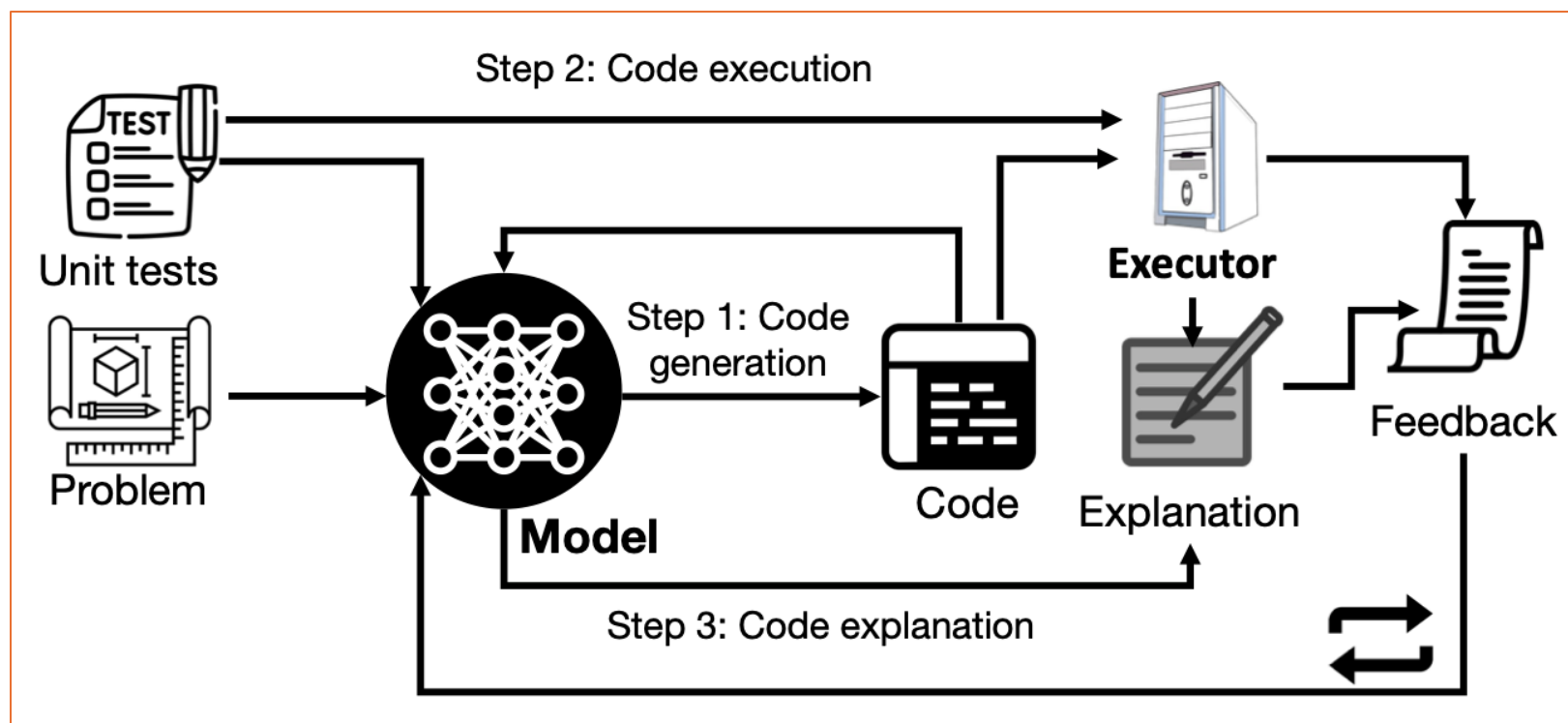
C++ (using GoogleTest)

TEACHING LARGE LANGUAGE MODELS TO SELF-DEBUG

Xinyun Chen¹ Maxwell Lin² Nathanael Schärli¹ Denny Zhou¹

¹ Google DeepMind ² UC Berkeley

{xinyunchen,schaerli,dennyzhou}@google.com, mxlin@berkeley.edu



TEACHING LARGE LANGUAGE MODELS TO SELF-DEBUG

Xinyun Chen¹ Maxwell Lin² Nathanael Schärli¹ Denny Zhou¹

¹ Google DeepMind ² UC Berkeley

{xinyunchen,schaerli,dennyzhou}@google.com, mxlin@berkeley.edu

Simple Feedback	Unit Test (UT) Feedback	Unit Test + Explanation (+Expl.)	Unit Test + Trace (+Trace)
Below are C++ programs with incorrect Python translations. Correct the translations using the provided feedback.	Below are C++ programs with incorrect Python translations. Correct the translations using the provided feedback.	Below are C++ programs with incorrect Python translations. <i>Explain the original code, then explain the translations line by line</i> and correct them using the provided feedback.	Below are C++ programs with incorrect Python translations. Using the provided feedback, <i>trace through the execution of the translations to determine what needs to be fixed</i> , and correct the translations.
[Original Python]	[Original Python]	[C++ Explanation]	[Original Python]
[Revised Python #1]	[Revised Python #1]	[Original Python]	[Trace]
[Revised Python #2]	[Revised Python #2]	[Python Explanation]	[Revised Python #1]
...	...	[UT Feedback]	[UT Feedback]
		[Revised Python #1]	[Trace]
		[Python Explanation]	[Revised Python #2]
		[UT Feedback]	...
		[Revised Python #2]	
		[Python Explanation]	
		...	

[Original Python] =

```
def count_trailing_zeroes_factorial_number(n):  
    cnt = 0  
    i = 5  
    while ((n / i) >= 1):  
        cnt = (cnt + (n / i))  
        i *= 5  
    return cnt
```

[Simple Feedback] =

The above Python translation does not do the same thing as the C++ code. Correct the Python translation.

[UT Feedback] =

The Python translation does not do the same thing as the C++ code. These are the results of failed unit tests that test whether the Python translation's outputs match the C++ program's outputs:

Failed: assert

```
count_trailing_zeroes_factorial_number(9) == 1
```

Actual Result: 1.8

Correct the translation.

[Revised Python #n] =

```
def count_trailing_zeroes_factorial_number(n):  
    cnt = 0  
    i = 5  
    while ((n / i) >= 1):  
        cnt = (cnt + (n // i))  
        i *= 5  
    return cnt
```

MBPP	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	61.4	67.6	72.8	47.2
Simple	68.2	70.8	78.8	50.6
UT	69.4	72.2	80.6	52.2
+ Expl.	69.8	74.2	80.4	52.2
+ Trace.	70.8	72.8	80.2	53.2

Self-Debugging with Different Feedback Formats

Unit Test Trace Feedback

- Issue
 - Unit test feedback only covers the expected output
 - It does not shed light on the **Root Cause**
 - Demands further analysis
- Solution
 - Reason about program execution via the use of **Debugger**

```
parity_buggy.py > ...
1  def parity(x: int) -> int:
2      if x == 0:
3          return 1 # BUG
4
5      res = 0
6      while x:
7          res ^= (x & 1)
8          x >>= 1
9      return res
10
11 # ----- Tests (simple assertions) -----
12 assert parity(0) == 0, "parity(0) should be 0 (even), but
13 assert parity(1) == 1, "1 -> 1 (binary), odd -> 1"
14 assert parity(2) == 1, "2 -> 10 (binary), one '1' -> 1"
```

```
liby99@hanabi ~/L/P/m/lecture-7-demos> python3 parity_buggy.py
Traceback (most recent call last):
  File "/Users/liby99/Local/Projects/machine-programming/lecture-7-demos/parity_buggy.py",
    line 13, in <module>
      assert parity(0) == 0, "parity(0) should be 0 (even), but the function returns 1"
      ~~~~~
AssertionError: parity(0) should be 0 (even), but the function returns 1
```

Debugger

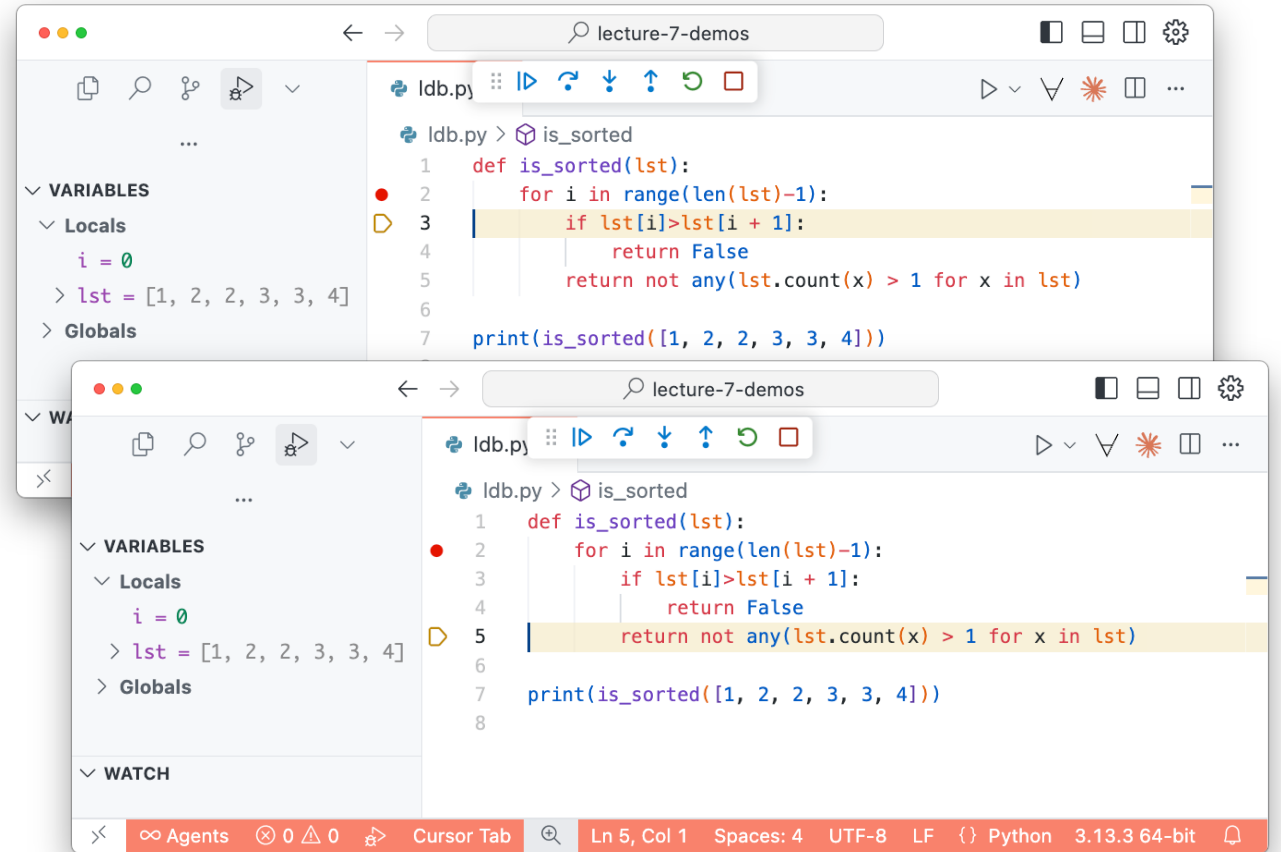
```
def is_sorted(lst):  
    for i in range(len(lst)-1):  
        if lst[i]>lst[i + 1]:  
            return False  
    return not any(lst.count(x) > 1 for x in lst)  
  
print(is_sorted([1, 2, 2, 3, 3, 4]))
```

```
liby99@hanabi ~/L/P/m/lecture-7-demos> python3 ldb.py  
False
```

Debugger

```
def is_sorted(lst):  
    for i in range(len(lst)-1):  
        if lst[i]>lst[i + 1]:  
            return False  
    return not any(lst.count(x) > 1 for x in lst)  
  
print(is_sorted([1, 2, 2, 3, 3, 4]))
```

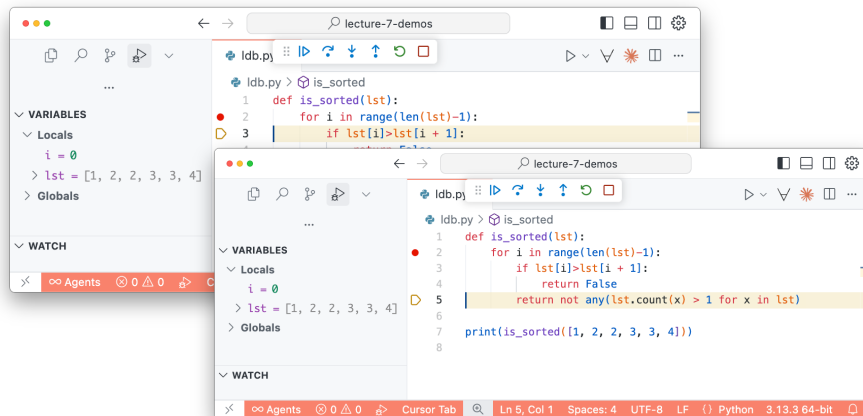
```
liby99@hanabi ~/L/P/m/lecture-7-demos> python3 ldb.py  
False
```



Debugger

```
def is_sorted(lst):  
    for i in range(len(lst)-1):  
        if lst[i]>lst[i + 1]:  
            return False  
    return not any(lst.count(x) > 1 for x in lst)  
  
print(is_sorted([1, 2, 2, 3, 3, 4]))
```

```
liby99@hanabi ~/L/P/m/lecture-7-demos> python3 ldb.py  
False
```



Program State Trace

line 2: `lst = [1, 2, 2, 3, 3, 4]`

line 3: `i = 0`
`lst = [1, 2, 2, 3, 3, 4]`

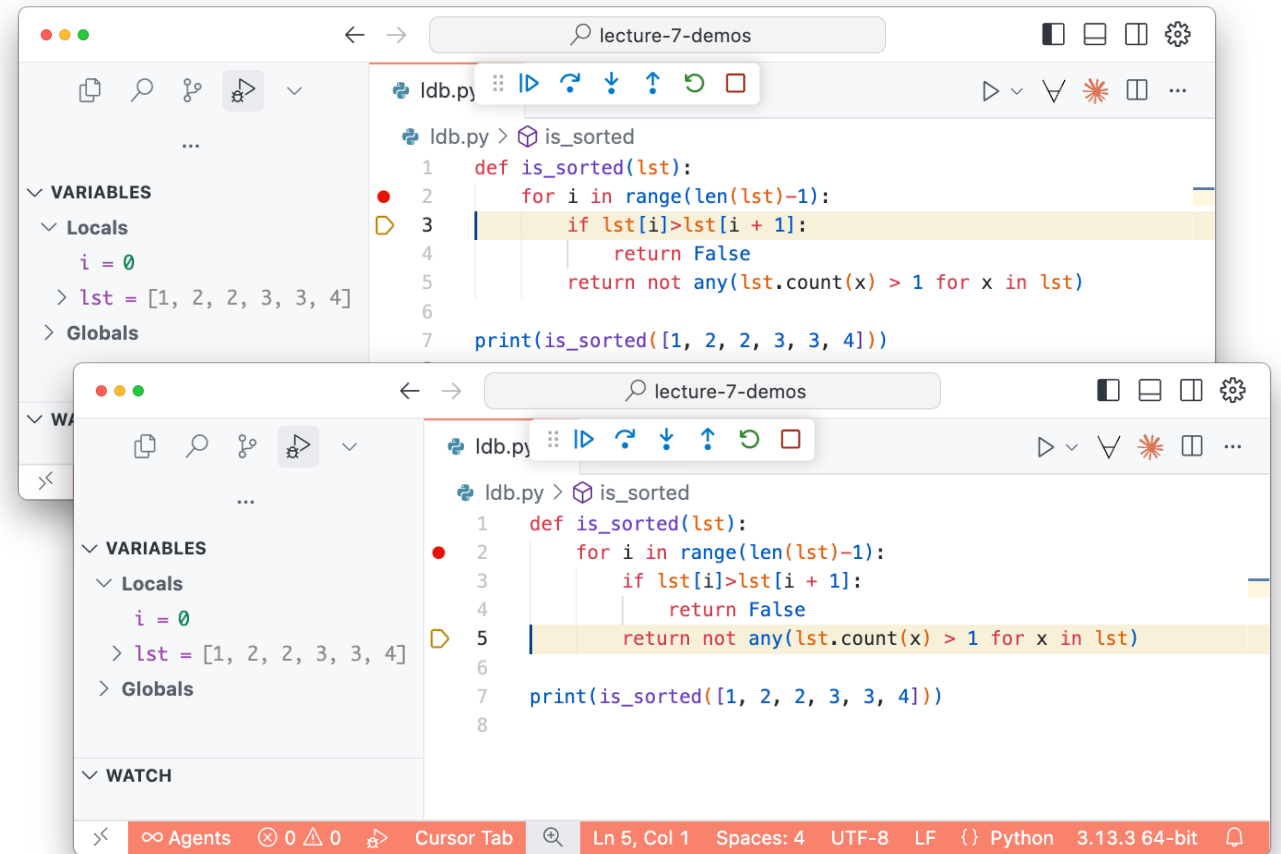
line 5: `i = 0`
`lst = [1, 2, 2, 3, 3, 4]`
`x = 1`

line 6: `c = 1`
`i = 0`
`lst = [1, 2, 2, 3, 3, 4]`
`x = 1`

...

Debugger for Feedback

- Issues
 - Too many intermediate states to step through
 - Loops may iterate many times resulting in very verbose feedback

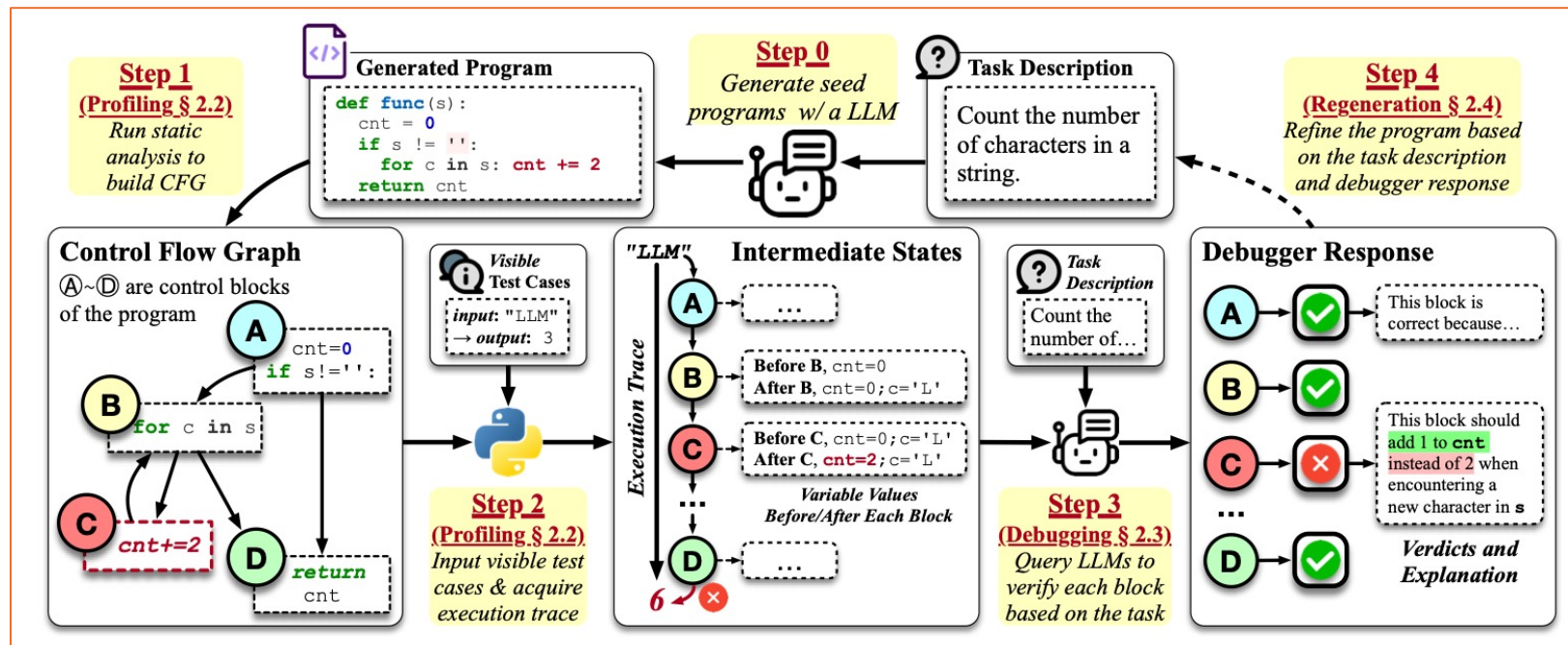


Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step

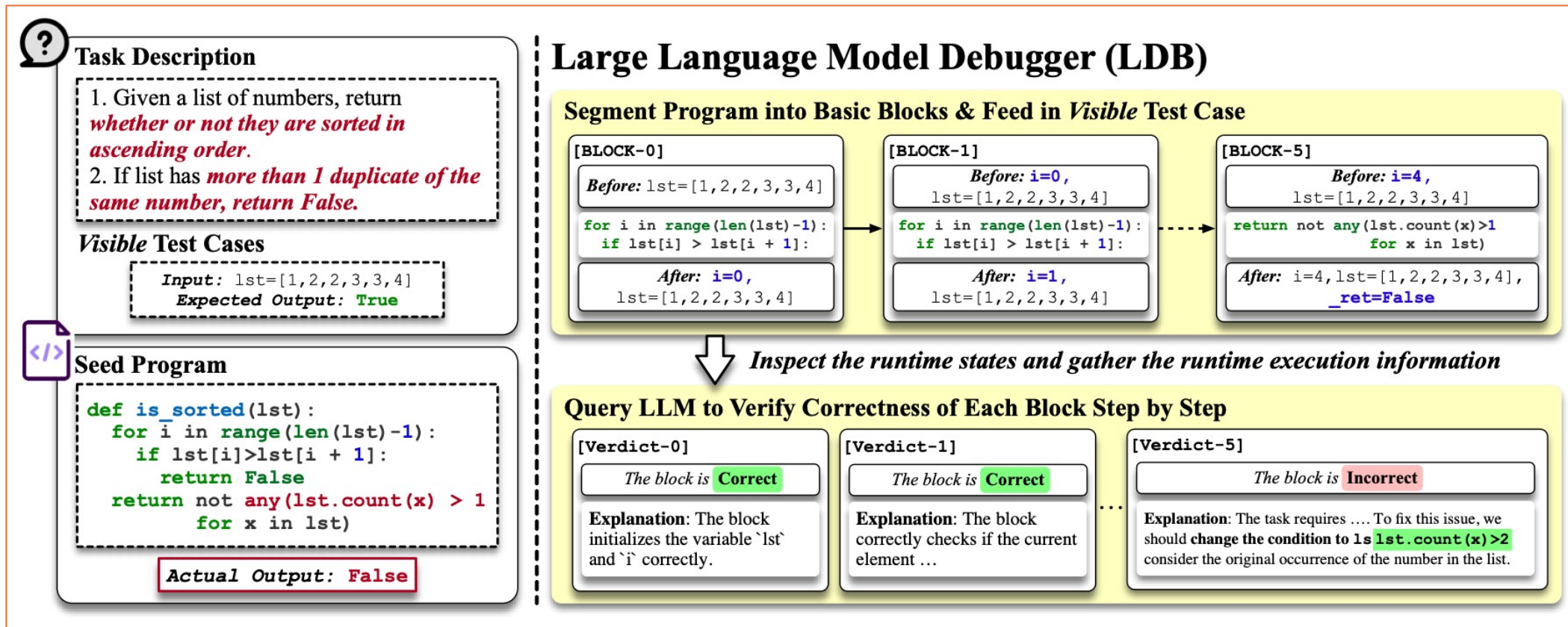
Li Zhong Zilong Wang[†] Jingbo Shang[†]
University of California, San Diego
{lizhong, zlwang, jshang}@ucsd.edu

Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step

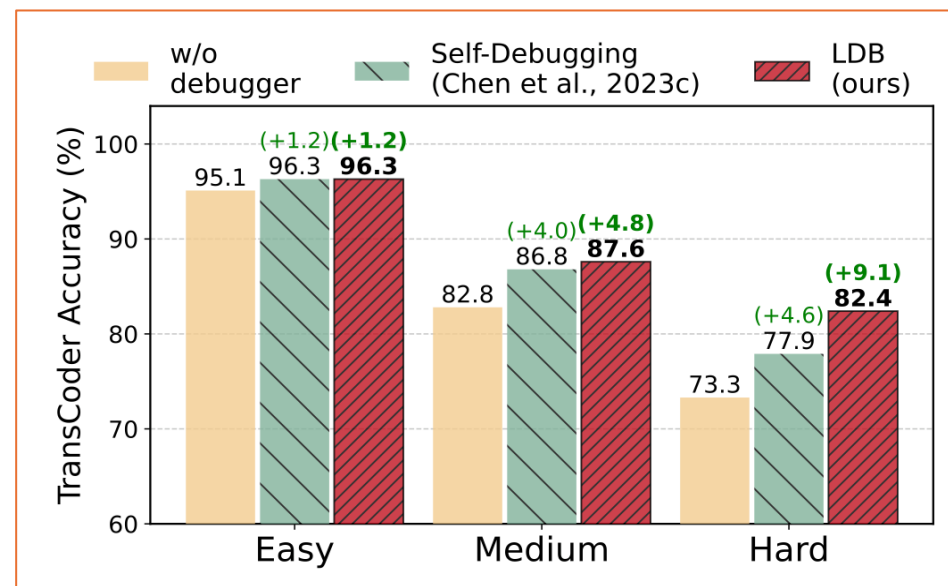
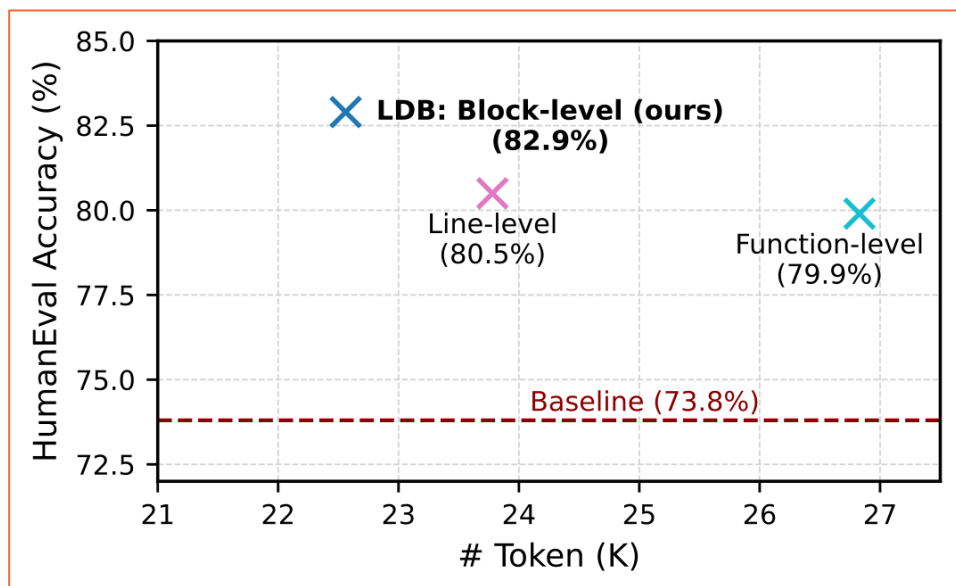
Li Zhong Zilong Wang[†] Jingbo Shang[†]
University of California, San Diego
{lizhong, zlwang, jshang}@ucsd.edu



Debugger for Feedback



Debugger for Feedback



More Unit Tests

- What if the provided unit tests are insufficient?
 - Does not cover edge cases
 - Does not guarantee correct solution

```
def is_palindrome(s: str) -> bool:
    """
    return True if s reads the same forwards and
    backwards. Ignore case and whitespace or
    punctuation.
    """
    return s == s[::-1] # Syntheized test case

class TestPalindrome(unittest.TestCase):
    def test_racecar_capitalized(self):
        # case-insensitive.
        self.assertTrue(is_palindrome("Racecar"))
```

Toward Trustworthy Neural Program Synthesis

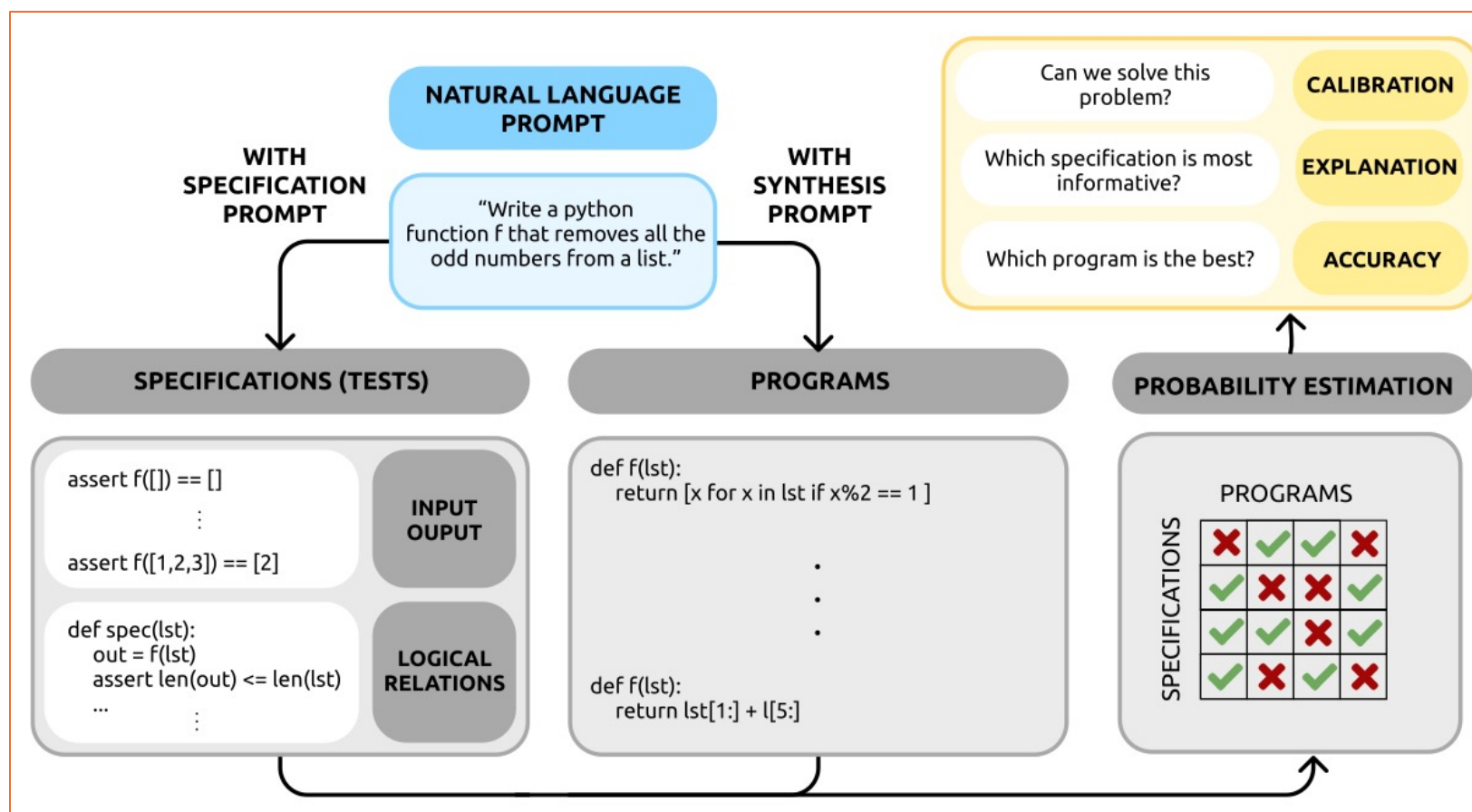
Wen-Ding Li^{*1}, Darren Key^{*1}, Kevin Ellis¹

¹Department of Computer Science, Cornell University, USA
wl678@cornell.edu, dyk34@cornell.edu, kellis@cornell.edu

Toward Trustworthy Neural Program Synthesis

Wen-Ding Li^{*1}, Darren Key^{*1}, Kevin Ellis¹

¹Department of Computer Science, Cornell University, USA
wl678@cornell.edu, dyk34@cornell.edu, kellis@cornell.edu



Toward Trustworthy Neural Program Synthesis

Wen-Ding Li^{*1}, Darren Key^{*1}, Kevin Ellis¹

¹Department of Computer Science, Cornell University, USA
wl678@cornell.edu, dyk34@cornell.edu, kellis@cornell.edu

Generating Program

```
def sub_list(nums1 : list, nums2 : list) -> list:
    """
    Write a function to subtract two lists element-wise.
    """
    return list(map(lambda x, y: x-y, nums1, nums2))
```

Generating Input-Output Specification

```
def sub_list(nums1 : list, nums2 : list) -> list:
    """
    Write a function to subtract two lists element-wise.
    """
    pass # To-do: implement

# Check if sub_list works
assert sub_list([2, 3, 1], [1, 1, 1]) == [1, 2, 0])
```

Toward Trustworthy Neural Program Synthesis

Generating Program

```
def sub_list(nums1 : list, nums2 : list) -> list:
    """
    Write a function to subtract two lists element-wise.
    """
    return list(map(lambda x, y: x-y, nums1, nums2))
```

Generating Input-Output Specification

```
def sub_list(nums1 : list, nums2 : list) -> list:
    """
    Write a function to subtract two lists element-wise.
    """
    pass # To-do: implement

# Check if sub_list works
assert sub_list([2, 3, 1], [1, 1, 1]) == [1, 2, 0])
```

Generating Logical-Relation Specification

```
[...FEWSHOT EXAMPLES...]
# Problem 3
# Write a function to subtract two lists element-wise.
def sub_list(nums1,nums2):
    pass # To-do: implement
# Test 3
def test_sub_list(nums1 : list, nums2 : list):
    """
    Given two lists `nums1` and `nums2`, test whether function `sub_list` is implemented correctly.
    """
    output_list = sub_list(nums1, nums2)
    # check if the length of the output list is the same as the lengths of the input lists
    assert len(output_list) == len(nums1) == len(nums2)
    # check if the output list has the expected elements
    for i in range(len(output_list)):
        assert output_list[i] == nums1[i] - nums2[i]

# run the testing function `test_sub_list` on a new testcase
test_sub_list([1, 2, 3, 4], [10, 9, 8, 7])
```

Property Based Testing

Verification of Program with a Specification

{Pre-condition} Program {Post-condition}

$$\{P\} c \{Q\}$$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

Property Based Testing

- Issues with verification with functional specifications:
 - Very hard to specify a **complete** functional specification
 - Pre- and post-conditions sometimes require **special constructs**
 - Requires a **formal theorem prover** / solver to verify

Verification of Program with a Specification

{Pre-condition} Program **{Post-condition}**

{P} c **{Q}**

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$\forall x, P(x) \Rightarrow Q(c(x))$

Property Based Testing

- Issues with verification with functional specifications:
 - Very hard to specify a **complete** functional specification
 - Pre- and post-conditions sometimes require **special constructs**
 - Requires a **formal theorem prover** / solver to verify
- Solution: **Property Based Testing!**

Verification of Program with a Specification

$\{\text{Pre-condition}\} \text{ Program } \{\text{Post-condition}\}$

$\{P\} c \{Q\}$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$\forall x, P(x) \Rightarrow Q(c(x))$

Property Based Testing

- Issues with verification with functional specifications:
 - Very hard to specify a **complete** functional specification
 - Pre- and post-conditions sometimes require **special constructs**
 - Requires a **formal theorem prover / solver** to verify
- Solution: **Property Based Testing!**

Property-Based Testing in Practice

Harrison Goldstein
University of Pennsylvania
Philadelphia, PA, USA
hgo@seas.upenn.edu

Joseph W. Cutler
University of Pennsylvania
Philadelphia, PA, USA
jwc@seas.upenn.edu

Daniel Dickstein
Jane Street
New York, NY, USA
ddickstein@janestreet.com

Benjamin C. Pierce
University of Pennsylvania
Philadelphia, PA, USA
bcpierce@seas.upenn.edu

Andrew Head
University of Pennsylvania
Philadelphia, PA, USA
head@seas.upenn.edu

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

P $x:\text{Int}$

```
int abs(int x) {  
    int y;  
    if (x >= 0) // B1  
        y = x; // B2  
    else  
        y = -x; // B3  
    return y; // B4  
}
```

Q $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

P $x:\text{Int}$

```
int abs(int x) {  
    int y;  
    if (x >= 0) // B1  
        y = x; // B2  
    else  
        y = -x; // B3  
    return y; // B4  
}
```

c

Q $(y = -x \vee y = x) \wedge y \geq 0$

```
bool abs_property(int x, int y) {  
    return (x == y || x == -y) &&  
           y >= 0;  
}
```

Property of abs encoded as Boolean function (predicate)

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

P $x: \text{Int}$

```
int abs(int x) {  
    int y;  
    if (x >= 0) // B1  
        y = x; // B2  
    else  
        y = -x; // B3  
    return y; // B4  
}
```

Q $(y = -x \vee y = x) \wedge y \geq 0$

```
bool abs_property(int x, int y) {  
    return (x == y || x == -y) &&  
           y >= 0;  
}
```

Property of abs encoded as Boolean function (predicate)

```
for (int i = 0; i < 1000; i++) {  
    int x = random_integer();  
    int y = abs(x);  
    assert(abs_property(x, y));  
}
```

Perform random testing with the property

Verification as Feedback

- Issues with verification with functional specifications:
 - Very hard to specify a **complete** functional specification
 - Pre- and post-conditions sometimes require **special constructs**
 - Requires a **formal theorem prover** / solver to verify

Verification of Program with a Specification

{Pre-condition} Program {Post-condition}

$$\{P\} c \{Q\}$$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

Verified Code Transpilation with LLMs

Sahil Bhatia¹ Jie Qiu Niranjan Hasabnis^{2*} Sanjit A. Seshia¹ Alvin Cheung¹

¹UC Berkeley ²Code Metal

{sahilbhatia, jieq, sseshia, akcheung}@berkeley.edu

niranjan@codemetal.ai

Verified Code Transpilation with LLMs

Sahil Bhatia¹ Jie Qiu Niranjana Hasabnis^{2*} Sanjit A. Seshia¹ Alvin Cheung¹
¹UC Berkeley ²Code Metal
{sahilbhatia, jieq, sseshia, akcheung}@berkeley.edu
niranjana@codemetal.ai

Code Transpilation

Language X $\longrightarrow$ Language Y

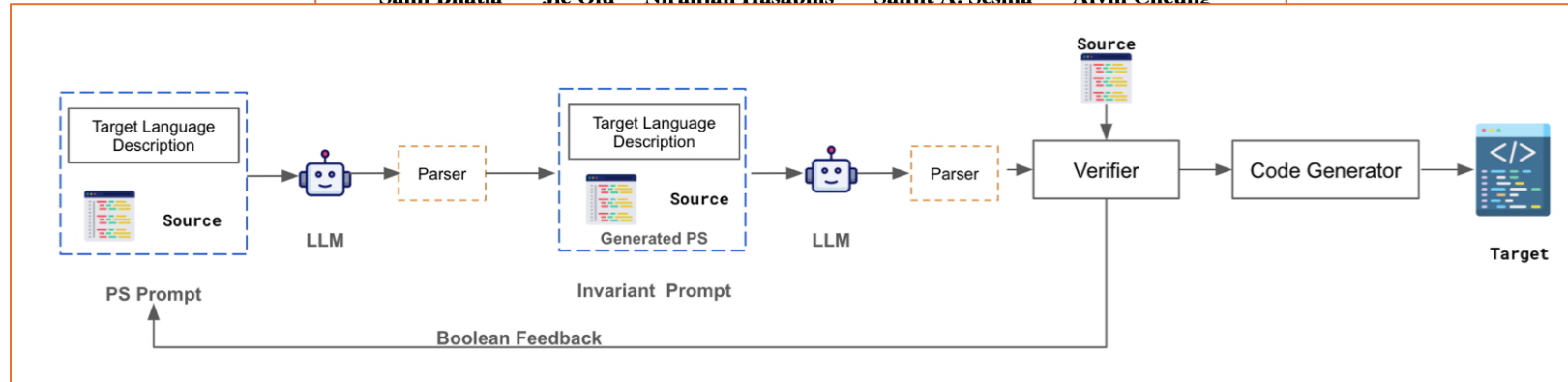
Given program $x \in X$ $\longrightarrow$ Synthesize $y \in Y$

Specification

$$\forall i, \text{eval}_X(x, i) = \text{eval}_Y(y, i)$$

Verified Code Transpilation with LLMs

Sahil Bhatia¹ Jie Oiu Niranian Hasabnis^{2*} Saniit A. Seshia¹ Alvin Cheung¹



Your task is to rewrite the given `test` C++ function using Apple MLX (array processing framework). The rewritten program should be semantically equivalent to the `test` function.

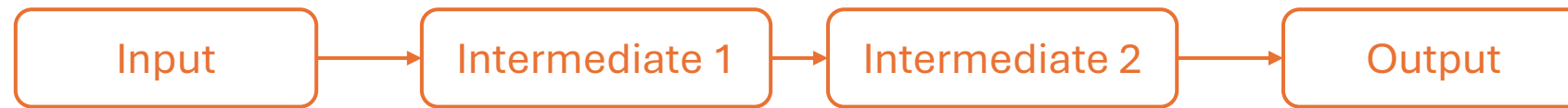
```
//test Function
vector<vector<int>> test(vector<vector<int>> b,
vector<vector<int>> a){
    vector<vector<int>> out;
    for (int row = 0; row < b.size(); row++) {
        vector<int> row_vec;
        for (int col = 0; col < b[0].size(); col++) {
            int pixel = (b[row][col] + a[row][col]) - 255;
            row_vec.push_back(pixel);
        }
        out.push_back(row_vec);
    }
    return out;}
```

```
#include <MLCompute/MLCompute.h>
mlcompute::Tensor test(mlcompute::Tensor b,
mlcompute::Tensor a) {
    auto out = mlcompute::Tensor::zeros_like(b);
    out = b + a - 255;
    return out;}
```

Summary of Iterative Refinement Feedback

- Compiler messages
- Linter messages
- Unit test report
 - Generating traces from unit test using debuggers
 - Simplifying traces via control flow graph
 - Generating more test cases
- Property based testing
- Verification result

From Multi-Step Prompting to Iterative Prompting



A known and fixed number of steps

- Zero-shot (0)
- Few-shot (0)
- Chain-of-thought (1)
- Self-Reflection (1)



A variable number of steps

- Self-Debug
- Self-Repair
- FunSearch
- Evolutionary Search
- Generational Search

IS SELF-REPAIR A SILVER BULLET FOR CODE GENERATION?

**Theo X. Olausson^{1, †}, Jeevana Priya Inala², Chenglong Wang²,
Jianfeng Gao², Armando Solar-Lezama¹**

¹MIT CSAIL ²Microsoft Research

IS SELF-REPAIR A SILVER BULLET FOR CODE GENERATION?

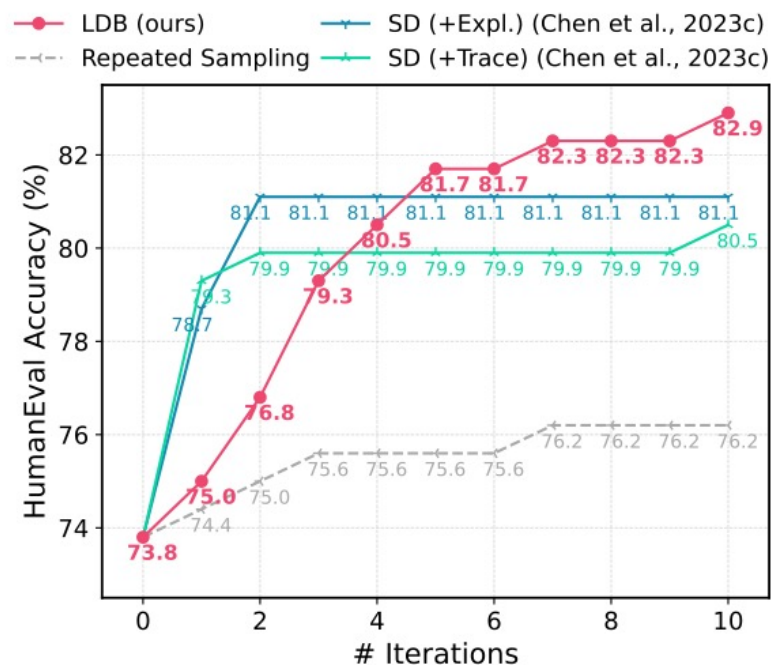
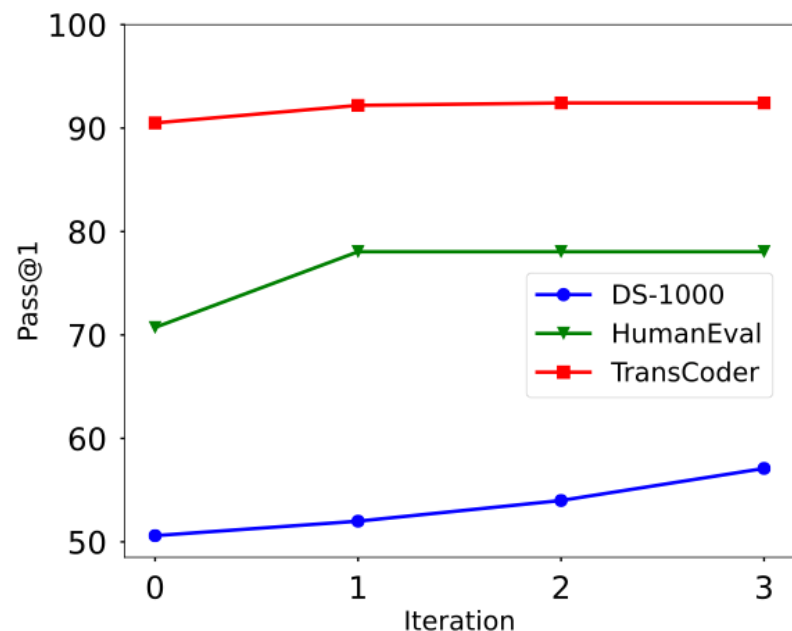
**Theo X. Olausson^{1, †}, Jeevana Priya Inala², Chenglong Wang²,
Jianfeng Gao², Armando Solar-Lezama¹**

¹MIT CSAIL ²Microsoft Research

- Self-repair is not a silver bullet: when the cost of repair is taken into account, we find several instances in which pass rates are higher or equally high with i.i.d. sampling (without repair), especially when the budget is small. We conjecture that this is because program generation and repair rates trend together, and many subtle factors influence which one will overpower the other for a given task (see Appendix [C](#)).

Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step

Li Zhong Zilong Wang[†] Jingbo Shang[†]
University of California, San Diego
{lizhong, zlwang, jshang}@ucsd.edu



SELF EVOLVE: A Code Evolution Framework via Large Language Models

Shuyang Jiang¹, Yuhao Wang¹, Yu Wang^{1,2*}
¹Shanghai Jiao Tong University
²Shanghai AI Laboratory
{jiangshuyang,colane,yuwangs@sjtu}@sjtu.edu.cn

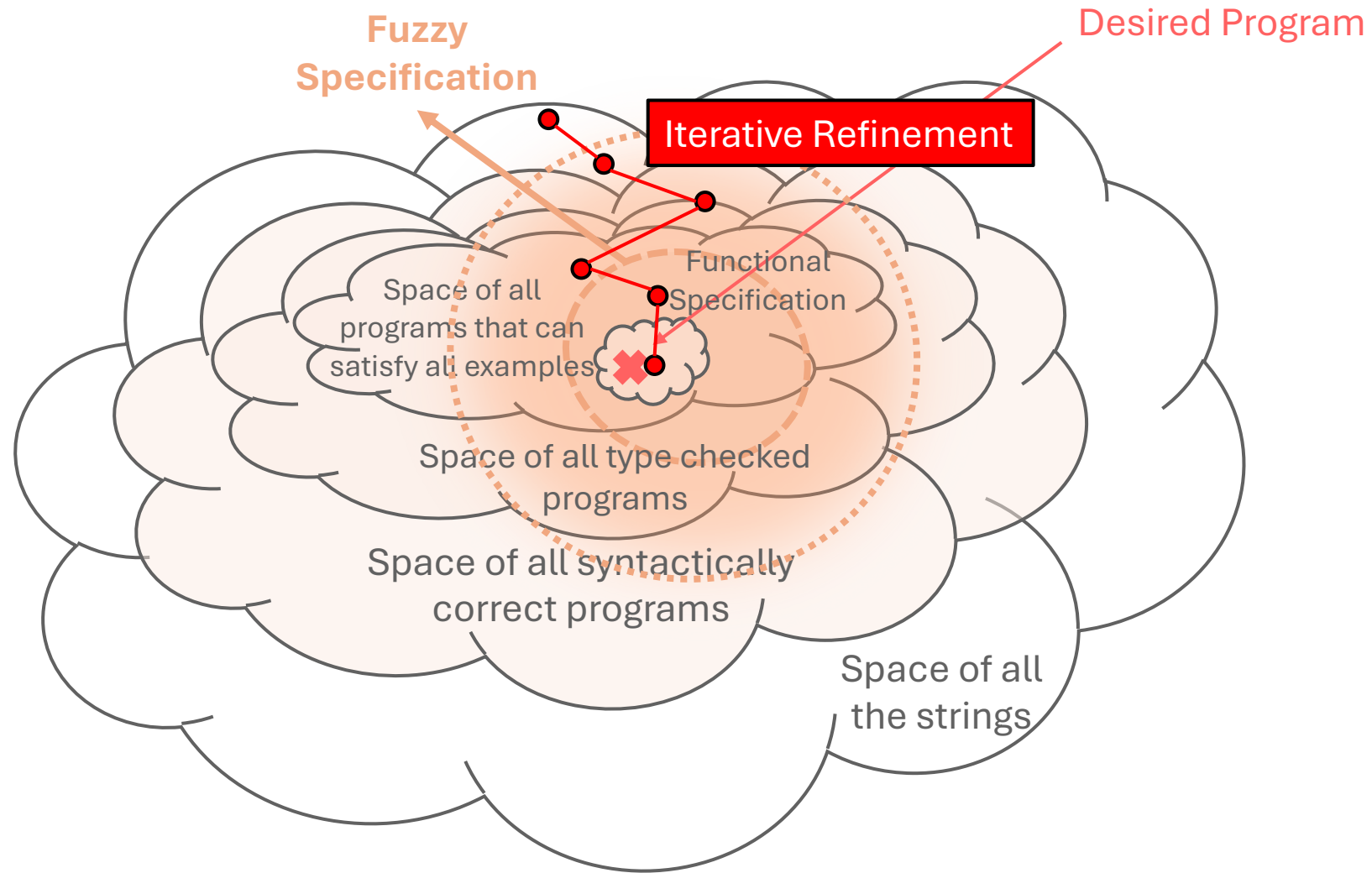
IS SELF-REPAIR A SILVER BULLET FOR CODE GENERATION?

- Self-repair is not a silver bullet: when the cost of repair is taken into account, we find several instances in which pass rates are higher or equally high with i.i.d. sampling (without repair), especially when the budget is small. We conjecture that this is because program generation and repair rates trend together, and many subtle factors influence which one will overpower the other for a given task (see Appendix C).
- Artificially boosting the quality of the feedback significantly improves the efficacy of self-repair. We replace Code Llama’s feedback with that produced by GPT-3.5 or GPT-4, and GPT-3.5’s feedback with that of GPT-4; in every case, the boosted configuration beats both the corresponding i.i.d. baseline and the corresponding self-repair configuration at all budgets. Furthermore, replacing GPT-4’s own explanations with those of a human programmer improves repair significantly, increasing the fraction of repaired programs which pass the tests by a factor of $1.58\times$ (from 33.3% to 52.6%).

Table 1: Success rate of repair with GPT-4’s explanations vs. with those of our human participants.

Difficulty	Introductory	Interview	Competition	Overall
GPT-4 Feedback	42.64%	19.33%	3.67%	33.30%
Human Feedback	62.21%	45.67%	14.67%	52.60%

High Level Picture



High Level Picture



Single-step Prompting

You

Write this function:
`def add(a, b):`

Language Model Output

`def add(a, b): return a + b`

Prompting with a Conversation

```
f"""
```

```
Write this function:
```

```
def add(a, b):
```

```
Output:
```

```
def add(a, b): return a + b
```

```
Write this function:
```

```
def reverse(s: str) -> str:
```

```
Output:
```

```
def reverse(s: str) -> str:
```

```
    output = ""
```

```
    for c in s:
```

```
        output = c + output
```

```
    return output
```

```
Write this function:
```

```
def is_palindrome(s: str) -> bool:
```

```
    """
```

You

Write this function:

```
def add(a, b):
```

Faked Language Model Output

```
def add(a, b): return a + b
```

You

Write this function:

```
def reverse(s: str)...
```

Faked Language Model Output

```
def reverse(s: str) -> str:
```

```
    output = ""
```

```
    for c in s:
```

```
        output = c + output
```

```
    return output
```

You

Write this function:

```
def is_palindrome(s: str)
```

Prompting with a Conversation

You

Write this function:
def add(a, b):

Faked Language Model Output

```
def add(a, b): return a + b
```

You

Write this function:
def reverse(s: str)...



Faked Language Model Output

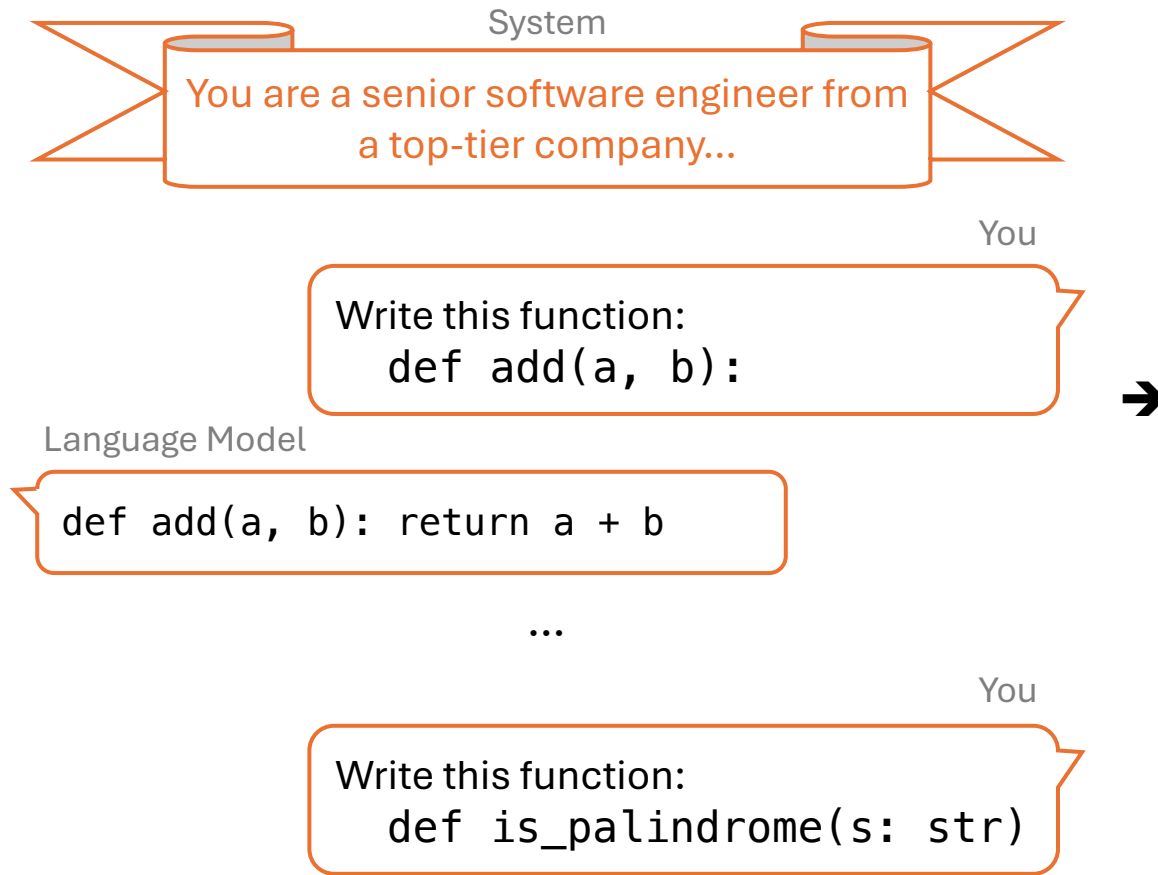
```
def reverse(s: str) -> str:
    output = ""
    for c in s:
        output = c + output
    return output
```

You

Write this function:
def is_palindrome(s: str)

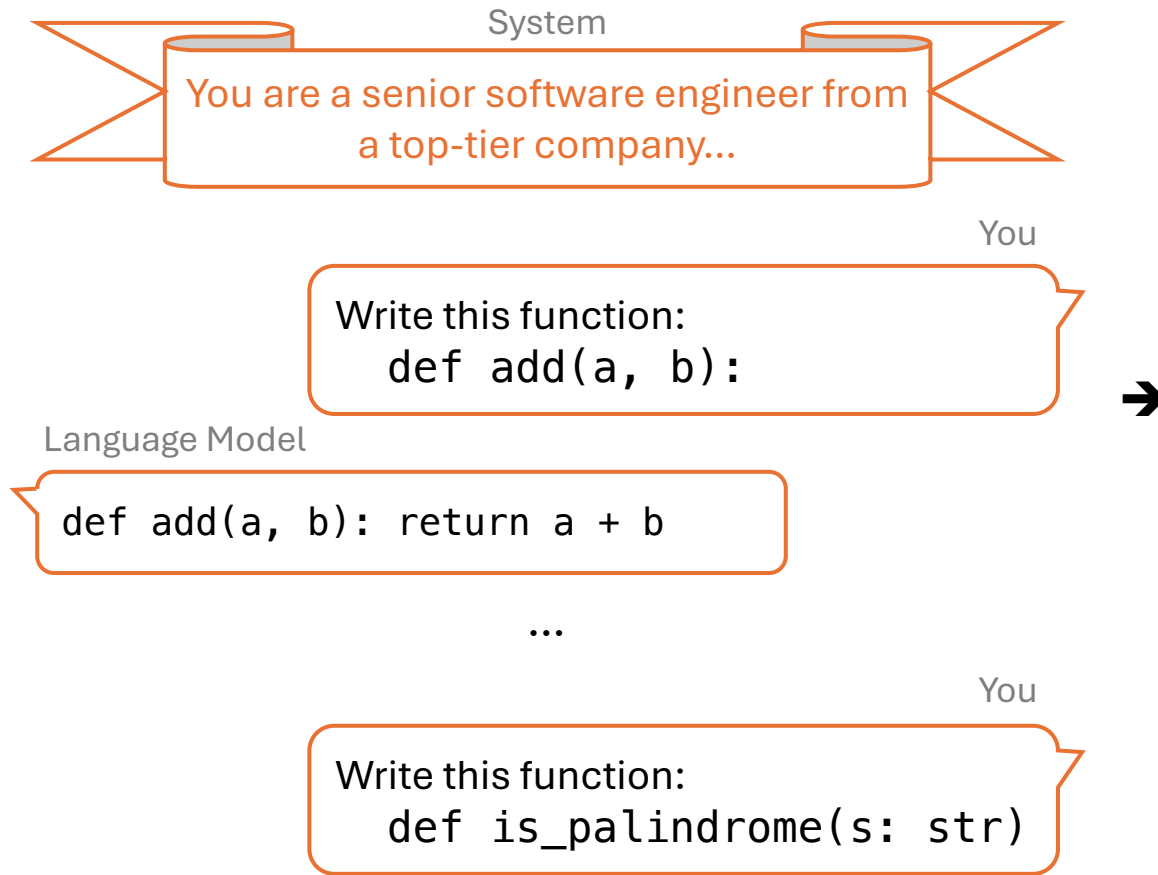
```
{
  "messages": [
    { "role": "user",
      "parts": ["Write this function:\ndef add(a, b):"] },
    { "role": "assistant",
      "parts": ["def add(a, b): return a + b"] },
    { "role": "user",
      "parts": ["Write this function:\ndef reverse(s: str)..."] },
    { "role": "assistant",
      "parts": ["def reverse(s: str) -> str:\n output = \"\"\n for c in s:\n output = c + output\n return output"] },
    { "role": "user",
      "parts": ["Write this function:\n def is_palindrome(s: str)"] }
  ]
}
```

Prompting with a Conversation



```
{
  "messages": [
    { "role": "system",
      "parts": ["You are a senior software engineer from..."] },
    { "role": "user",
      "parts": ["Write this function:\ndef add(a, b):"] },
    { "role": "assistant",
      "parts": ["def add(a, b): return a + b"] },
    { "role": "user",
      "parts": ["Write this function:\ndef reverse(s: str)..."] },
    { "role": "assistant",
      "parts": ["def reverse(s: str) -> str:\n output = \"\"\n
                for c in s:\n output = c + output\n
                return output"] },
    { "role": "user",
      "parts": ["Write this function:\n
                def is_palindrome(s: str)"] }
  ]
}
```

Prompting with a Conversation



```
{  
  "messages": [  
    { "role": "system",  
      "parts": ["You are a senior software engineer from..."] },  
    { "role": "user",  
      "parts": ["Write this function:\ndef add(a, b):"] },  
    { "role": "assistant",  
      "parts": ["def add(a, b): return a + b"] },  
    { "role": "user",  
      "parts": ["Write this function:\ndef reverse(s: str)..."] },  
    { "role": "assistant",  
      "parts": ["def reverse(s: str) -> str:\n output = \"\"\n for c in s:\n output = c + output\n return output"] },  
    { "role": "user",  
      "parts": ["Write this function:\n def is_palindrome(s: str)"] }  
  ]  
}
```

```
client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages=messages  
)
```

Prompting with a Conversation: Behind the Scene

Special Tokens denoting Roles

```
{
  "messages": [
    { "role": "system",
      "parts": ["You are a senior software engineer from..."] },
    { "role": "user",
      "parts": ["Write this function:\ndef add(a, b):"] },
    { "role": "assistant",
      "parts": ["def add(a, b): return a + b"] },
    { "role": "user",
      "parts": ["Write this function:\ndef reverse(s: str)..."] },
    { "role": "assistant",
      "parts": ["def reverse(s: str) -> str:\n output = \"\"\n for c in s:\n output = c + output\n return output"] },
    { "role": "user",
      "parts": ["Write this function:\ndef is_palindrome(s: str)"] }
  ]
}
```



```
<system>
You are a senior software engineer from...
<user>
Write this function:
  def add(a, b):
<assistant>
def add(a, b): return a + b
<user>
Write this function:
  def reverse(s: str)...
<assistant>
def reverse(s: str) -> str:
  output = ""
  for c in s:
    output = c + output
  return output
<user>
Write this function:
  def is_palindrome(s: str)
```

Chain-of-Thought Prompting

You

You are tasked to write this function:
`def min_cost_k_perio...(x, ...):`
Before writing it, please first think step-by-step.
analyze what needs to be done and sketch out a
piece of pseudo code...

Language Model (Actual Generation)

I will write the function
`min_cost_k_periodic_segmentation` to the
specified requirements. The problem appears
to be a dynamic programming challenge.

You

Good. Please go ahead and write the function.

Language Model (Actual Generation)

```
def min_cost_k_perio...(x, ...):  
    ...
```


Chain-of-Thought Prompting

You

You are tasked to write this function:
`def min_cost_k_perio...(x, ...):`
Before writing it, please first think step-by-step. analyze what needs to be done and sketch out a piece of pseudo code...

Language Model (Actual Generation)

I will write the function `min_cost_k_periodic_segmentation` to the specified requirements. The problem appears to be a dynamic programming challenge.

You

Good. Please go ahead and write the function.

Language Model (Actual Generation)

```
def min_cost_k_perio...(x, ...):  
    ...
```



```
<system>  
You are a senior software engineer from...  
<user>  
You are tasked to write this function:  
    def min_cost_k_perio...(x, ...):  
Before writing it, please first think step-  
by-step. analyze what needs to be done and  
sketch out a piece of pseudo code...  
<assistant>  
I will write the function  
min_cost_k_periodic_segmentation to the  
specified requirements. The problem appears  
to be a dynamic programming challenge.  
<user>  
Good. Please go ahead and write the  
function.
```

Self-Reflection Prompting

You

You are tasked to write this function:
`def min_cost_k_perio...(x, ...):`
Please write the function according to the specification and the input/output examples.

Language Model (Actual Generation)

```
def min_cost_k_perio...(x, ...): ...
```

You

Please now reflect what you have written. Try simulate what would happen if we pass in an example and see whether the result is expected.

Language Model (Actual Generation)

I will analyze the function that I just wrote...

You

Now please use your analysis to correct the function that you wrote; if nothing wrong, just copy and paste the function you had generated.

Self-Reflection Prompting

You

You are tasked to write this function:
def min_cost_k_perio...(x, ...):
Please write the function according to the
specification and the input/output examples.

Language Model (Actual Generation)

def min_cost_k_perio...(x, ...): ...

You

Please now reflect what you have written. Try
simulate what would happen if we pass in an
example and see whether the result is expected.

Language Model (Actual Generation)

I will analyze the function that I just wrote...

You

Now please use your analysis to correct the
function that you wrote; if nothing wrong, just
copy and paste the function you had generated.



```
<user>
You are tasked to write this function:
    def min_cost_k_perio...(x, ...):
Please write the function according to the
specification and the input/output
examples.
<assistant>
def min_cost_k_perio...(x, ...): ...
<user>
Please now reflect what you have written.
Try simulate what would happen if we pass
in an example and see whether the result is
expected.
<assistant>
I will analyze the function that I just
wrote...
<user>
Now please use your analysis to correct the
function that you wrote; if nothing wrong,
just copy and paste the function you had
generated.
<assistant>
...
```

Iterative Refinement

You

You are tasked to write this function:
`def min_cost_k_perio...(x, ...):`

Language Model

`def min_cost_k_perio...(x, ...): ...`

You (After running the test cases)

You passed 3/5 test cases; the failed test cases...

Language Model

`def min_cost_k_perio...(x, ...): ...`

You (After running the test cases)

You passed 4/5 test cases; the failed test cases...

Language Model

`def min_cost_k_perio...(x, ...): ...`

Iterative Refinement

You

You are tasked to write this function:
`def min_cost_k_perio...(x, ...):`

Language Model

`def min_cost_k_perio...(x, ...): ...`

You (After running the test cases)

You passed 3/5 test cases; the failed test cases...

Language Model

`def min_cost_k_perio...(x, ...): ...`

You (After running the test cases)

You passed 4/5 test cases; the failed test cases...

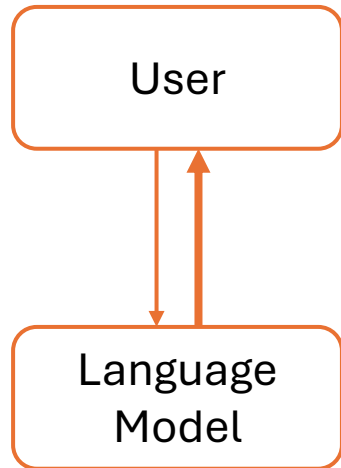
Language Model

`def min_cost_k_perio...(x, ...): ...`



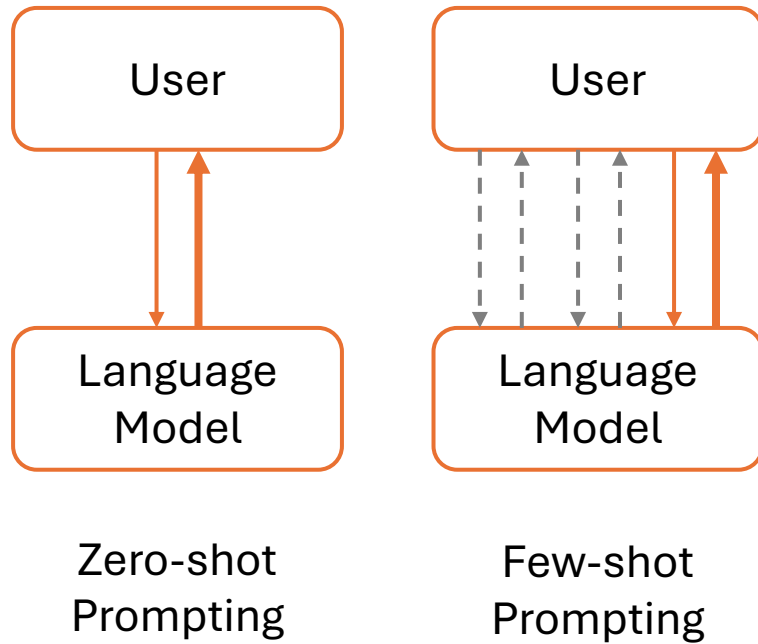
```
<user>
You are tasked to write this function:
  def min_cost_k_perio...(x, ...):
<assistant>
def min_cost_k_perio...(x, ...): ...
<user>
You passed 3/5 test cases; the failed test
cases...
<assistant>
def min_cost_k_perio...(x, ...): ...
<user>
You passed 4/5 test cases; the failed test
cases...
<assistant>
def min_cost_k_perio...(x, ...): ...
```

Prompting Pipelines

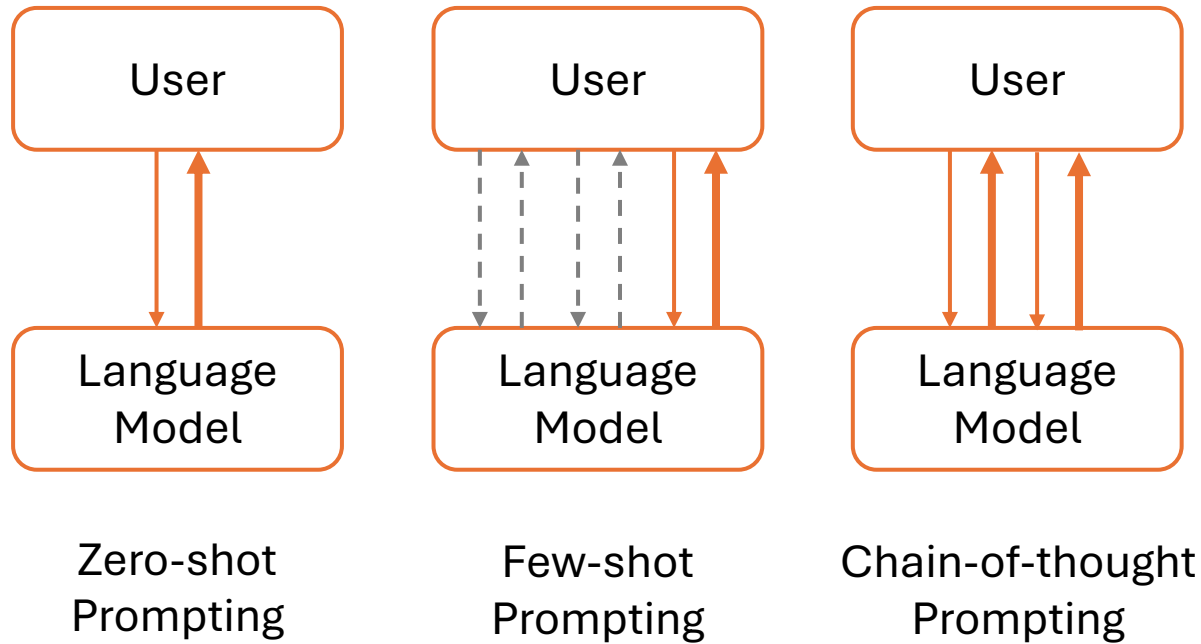


Zero-shot
Prompting

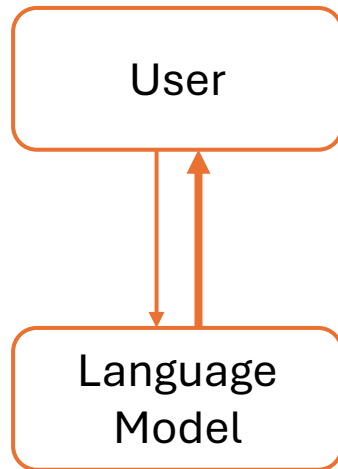
Prompting Pipelines



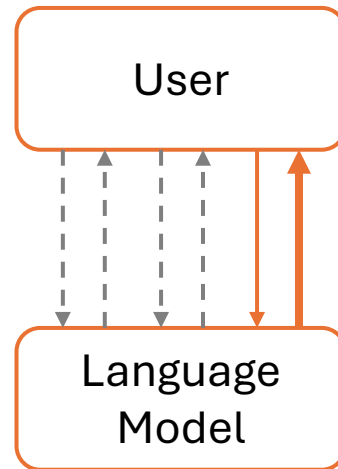
Prompting Pipelines



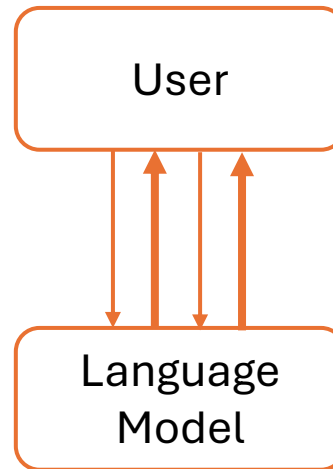
Prompting Pipelines



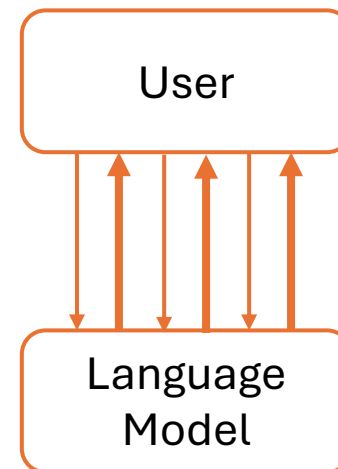
Zero-shot
Prompting



Few-shot
Prompting



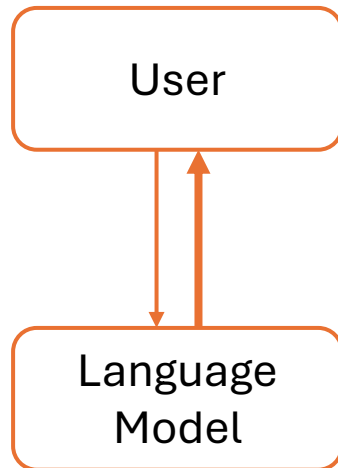
Chain-of-thought
Prompting



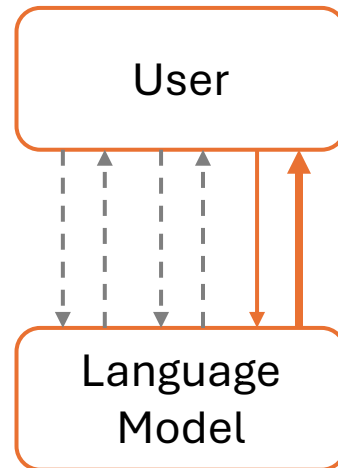
Self-Reflection
Prompting

Prompting Pipelines

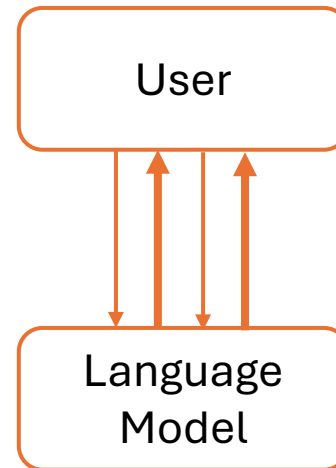
Compilers, Linters, Testers, Debuggers, Verifiers



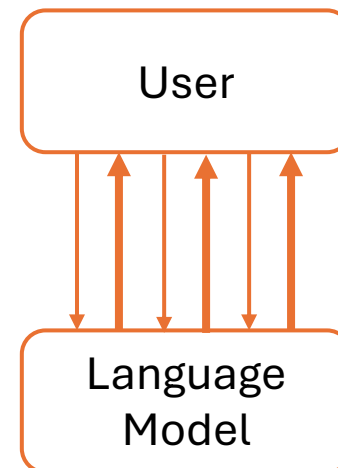
Zero-shot Prompting



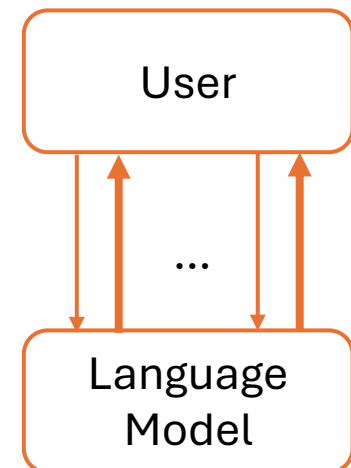
Few-shot Prompting



Chain-of-thought Prompting



Self-Reflection Prompting

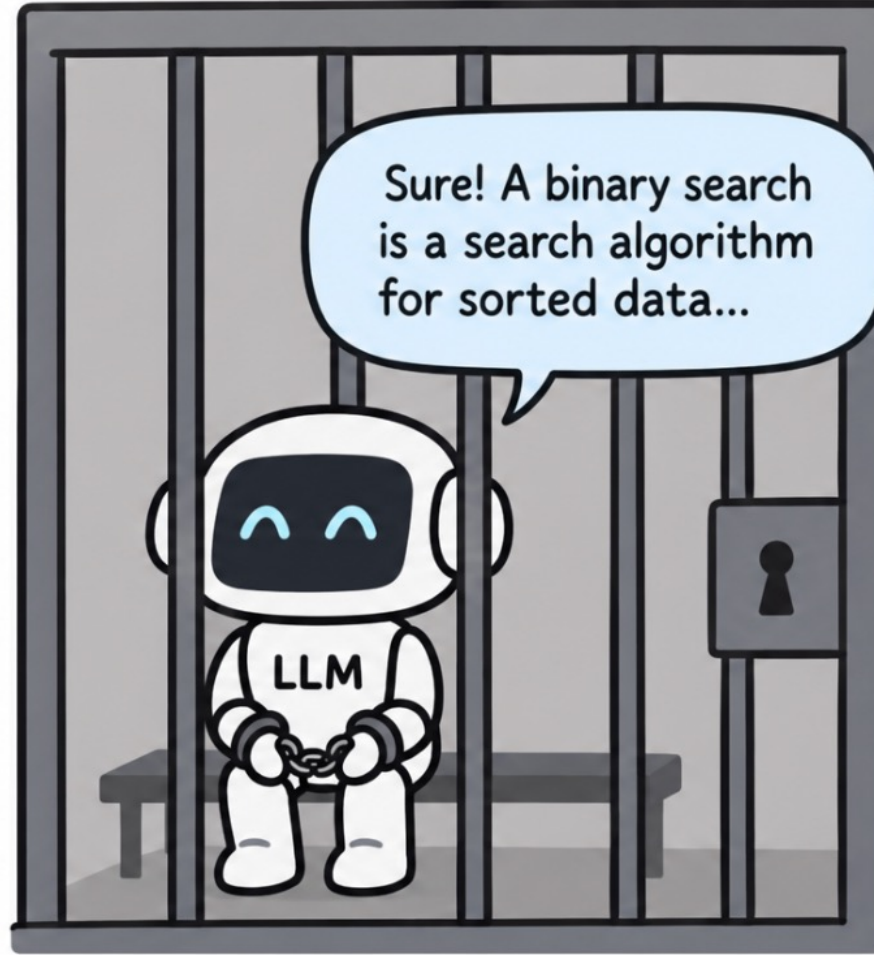


Iterative Refinement

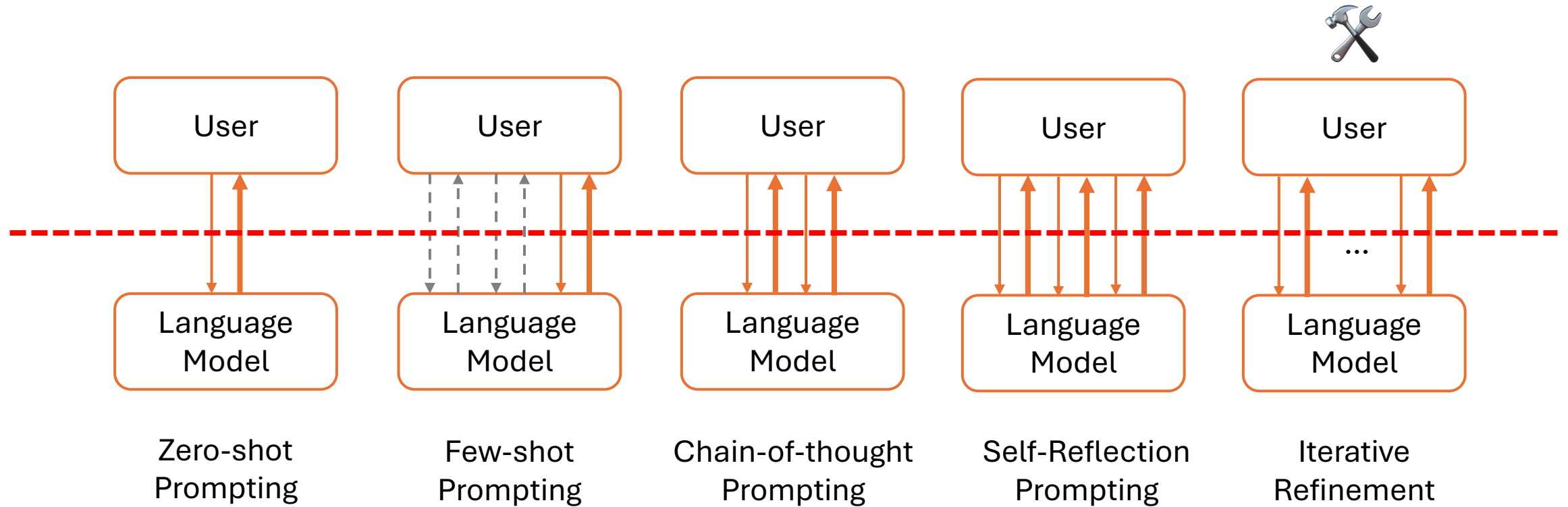
Hey LLM,
can you explain
what is a binary
search?



Sure! A binary search
is a search algorithm
for sorted data...

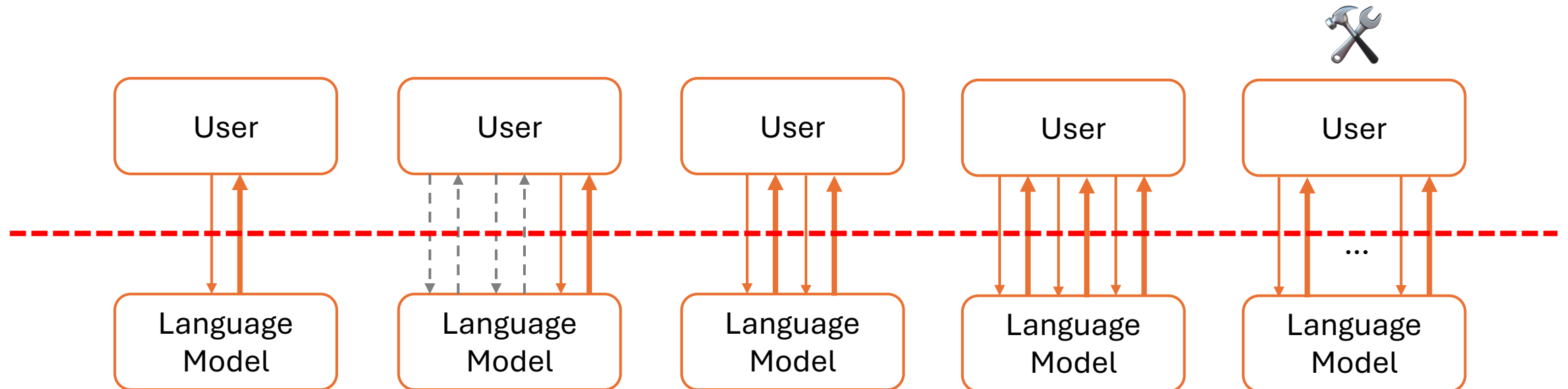


Prompting Pipelines



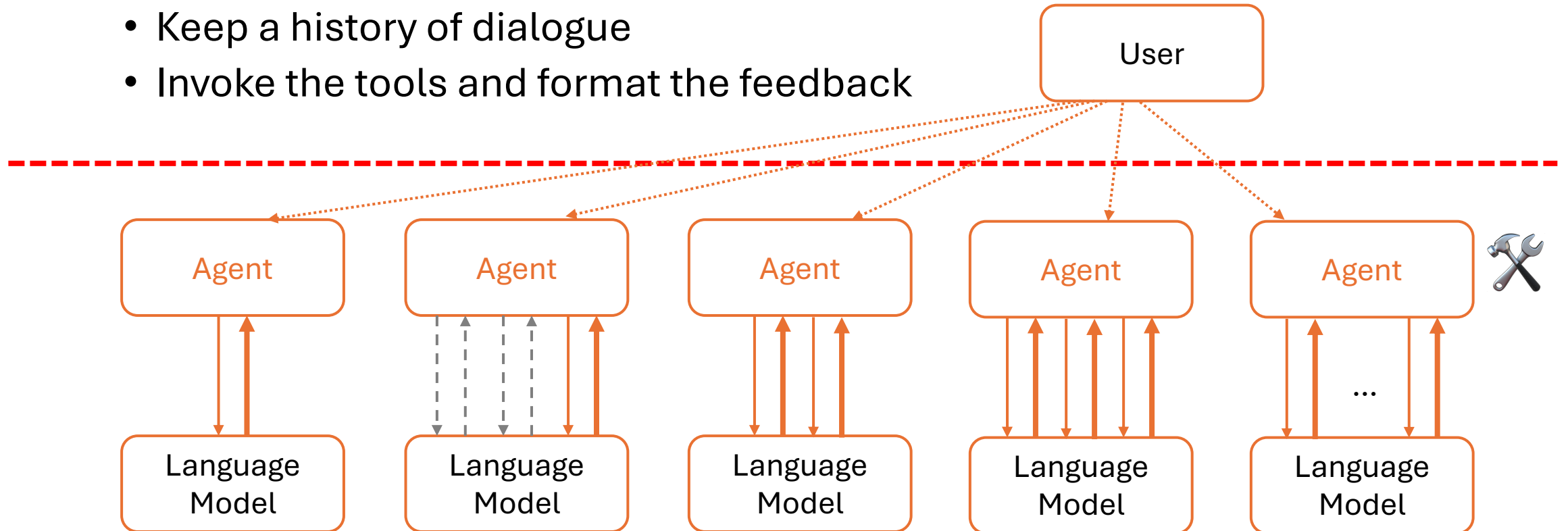
Prompting Pipelines

- User holds the responsibility to
 - Send requests to LLM
 - Keep a history of dialogue
 - Invoke the tools and format the feedback



Agentic Pipelines

- **Agent** holds the responsibility to
 - Send requests to LLM
 - Keep a history of dialogue
 - Invoke the tools and format the feedback

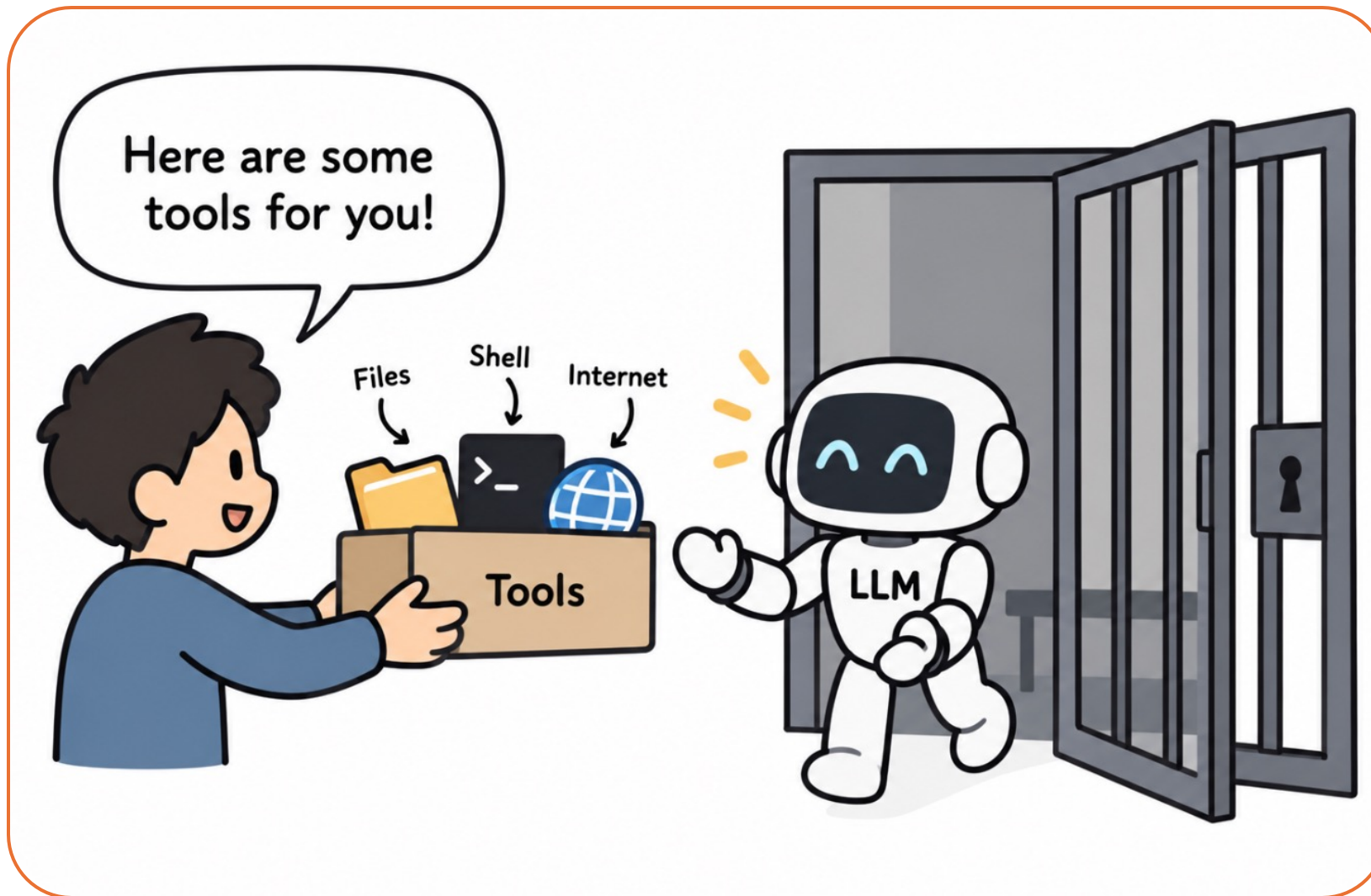


agency (noun)

- The capacity to act:
 - Definition: The ability of an individual or entity to **make choices** and **act** on them **independently**.
 - Example: “Children gradually develop their own agency in deciding what they want to study.”

LLM's agency (noun)

- The capacity to act:
 - Definition: The ability of an **LLM** to **make choices** and **act** on them **independently**.
 - Example: “**LLM** gradually develop their own agency in deciding what they want to study.”



Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)

Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)

LM

Harness

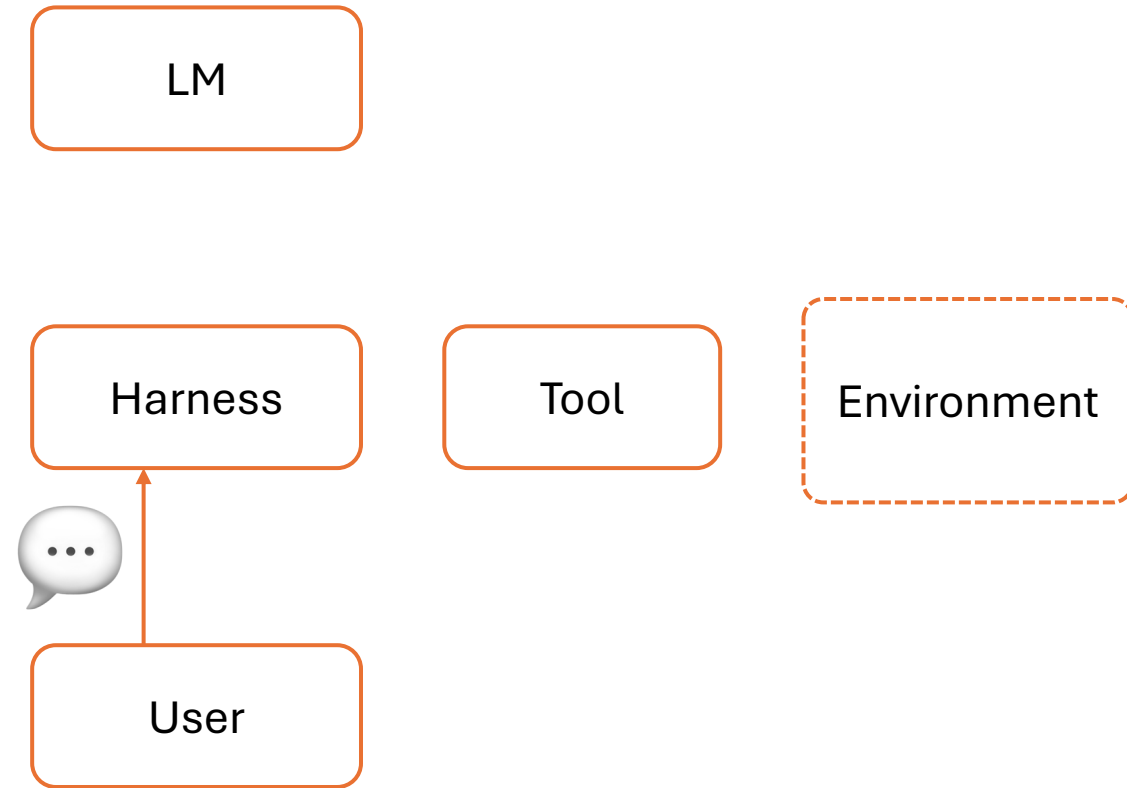
Tool

Environment

User

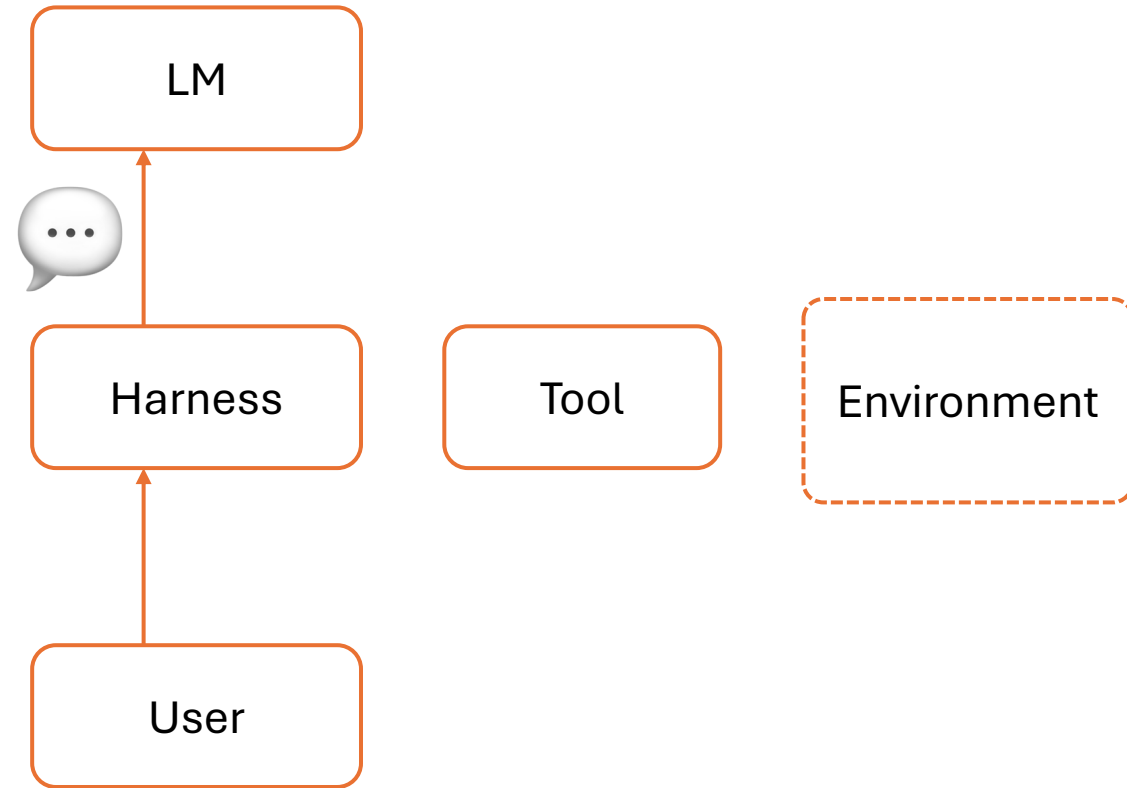
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



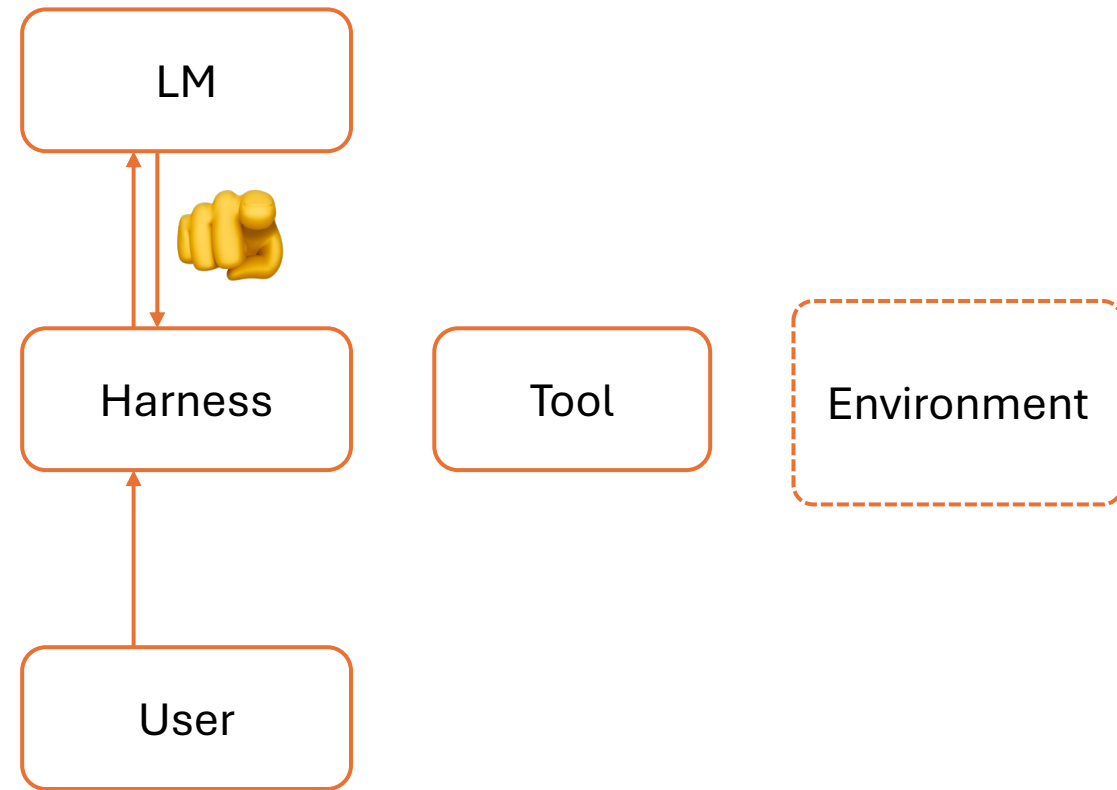
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



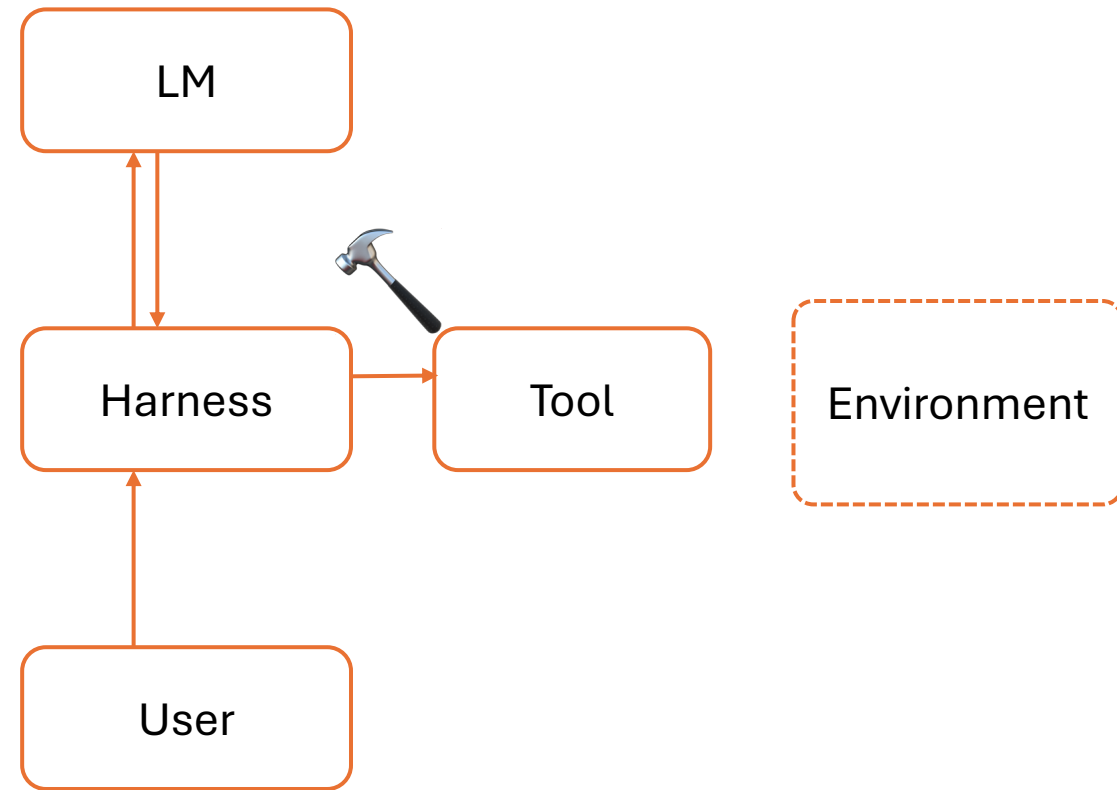
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



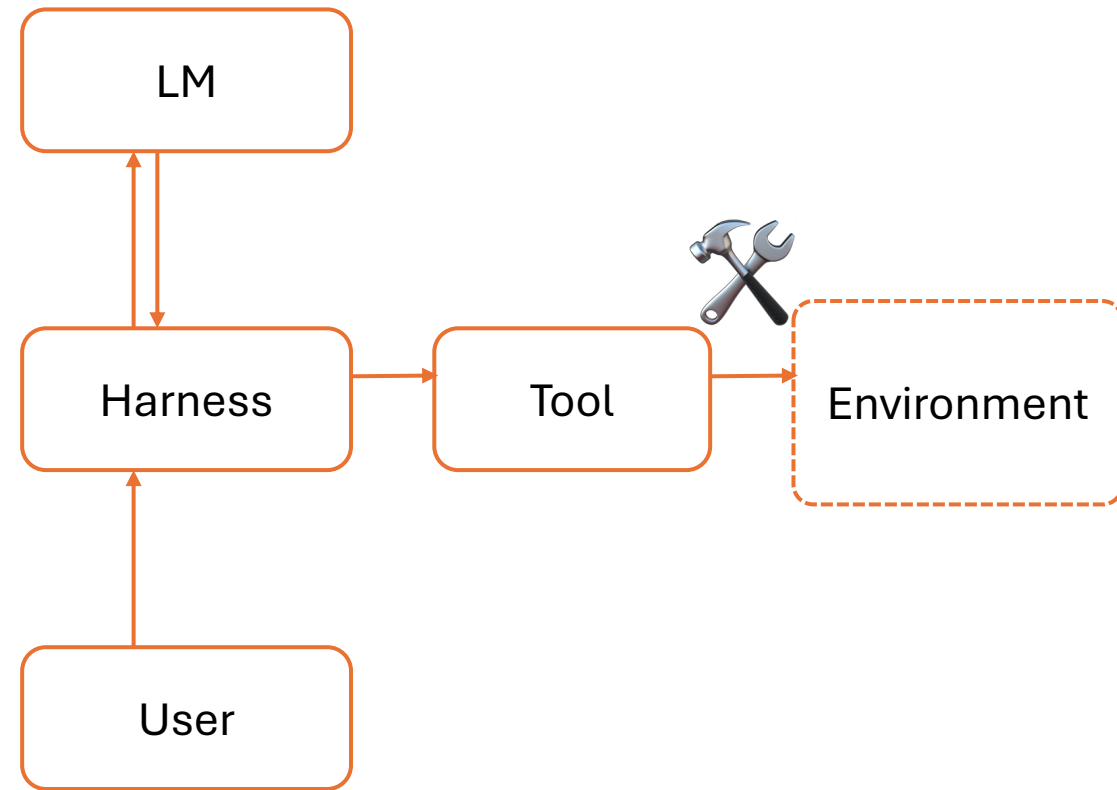
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



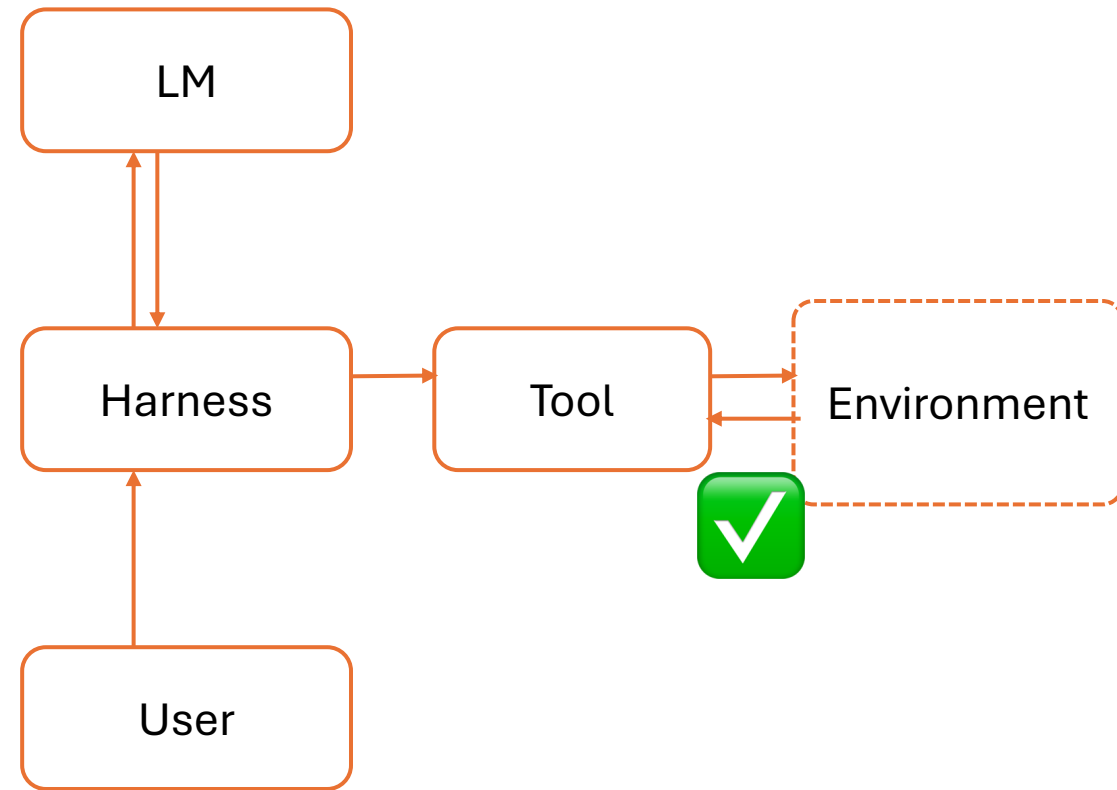
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



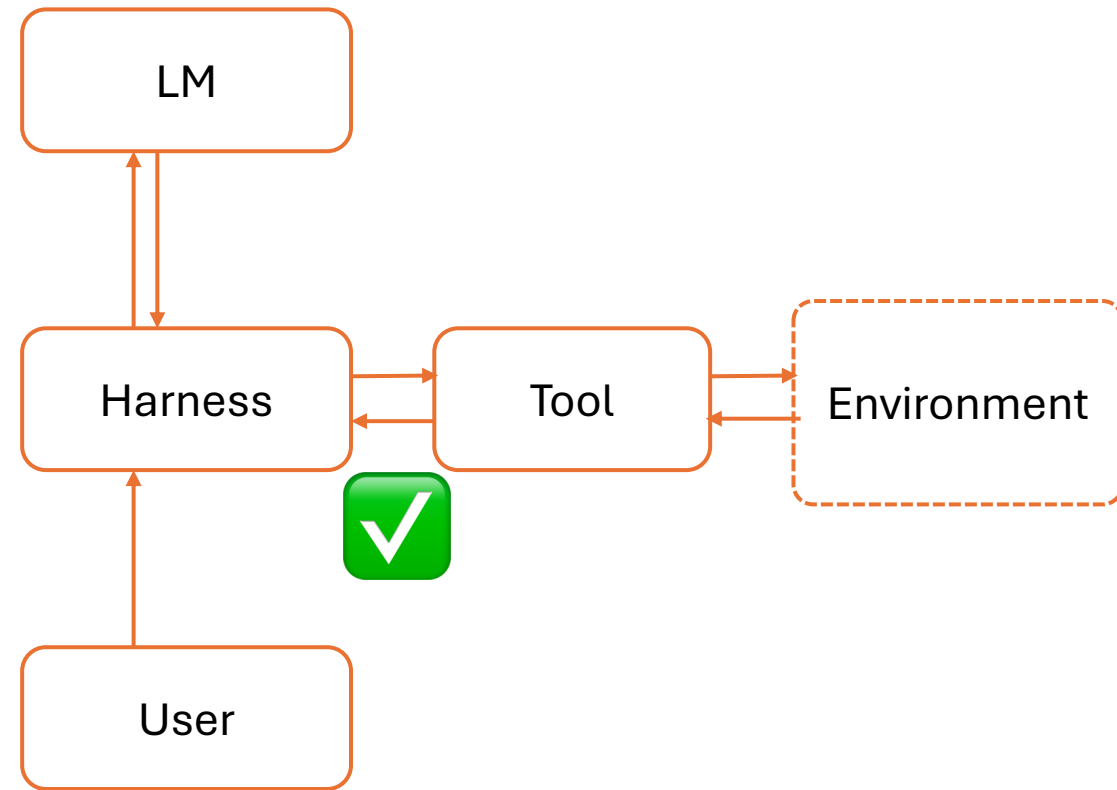
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



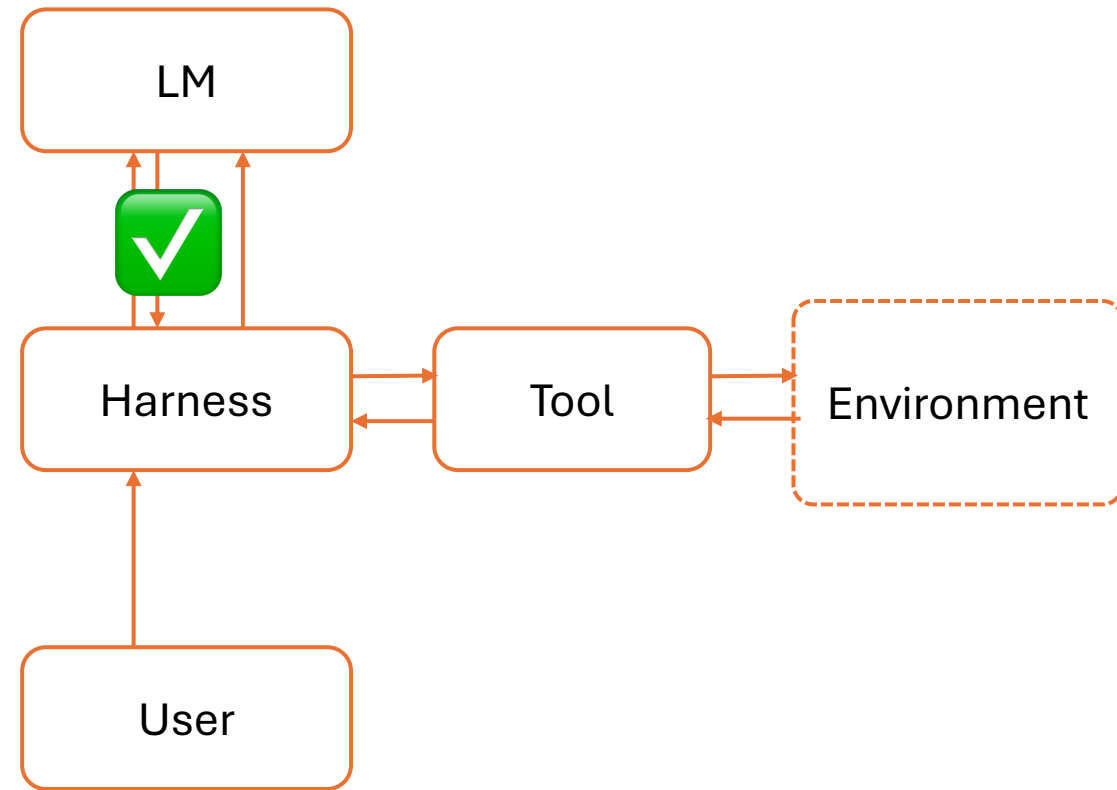
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



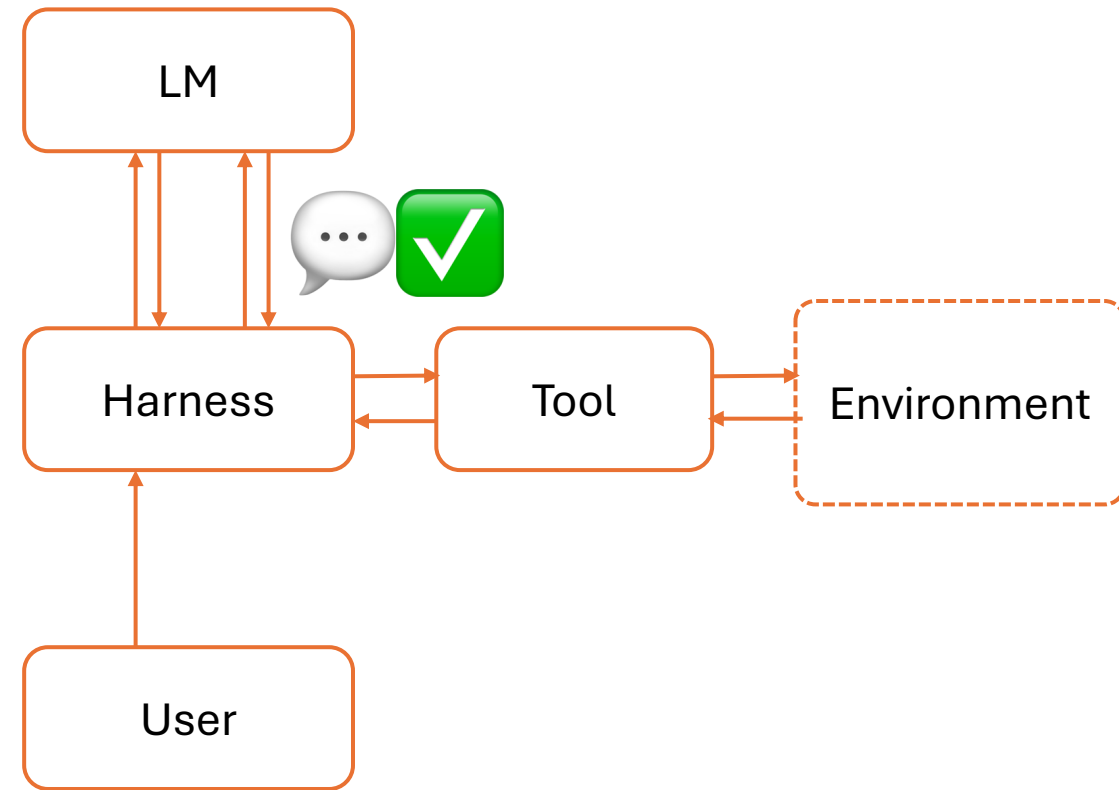
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



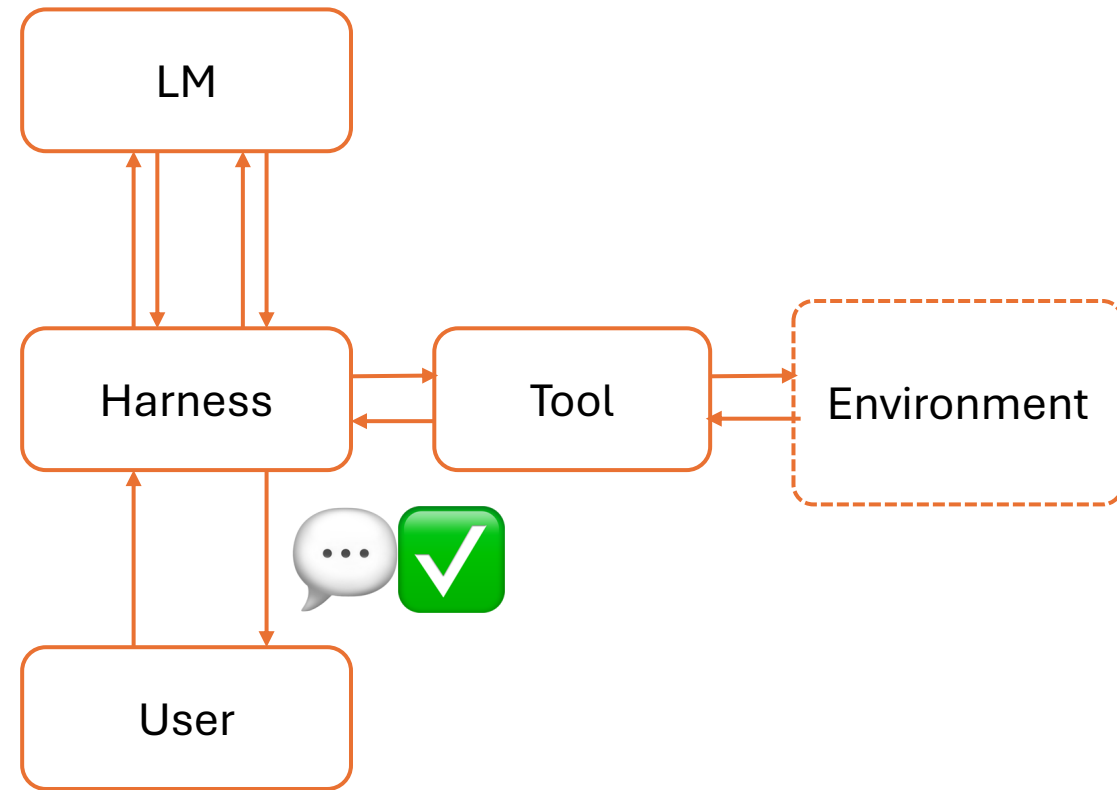
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



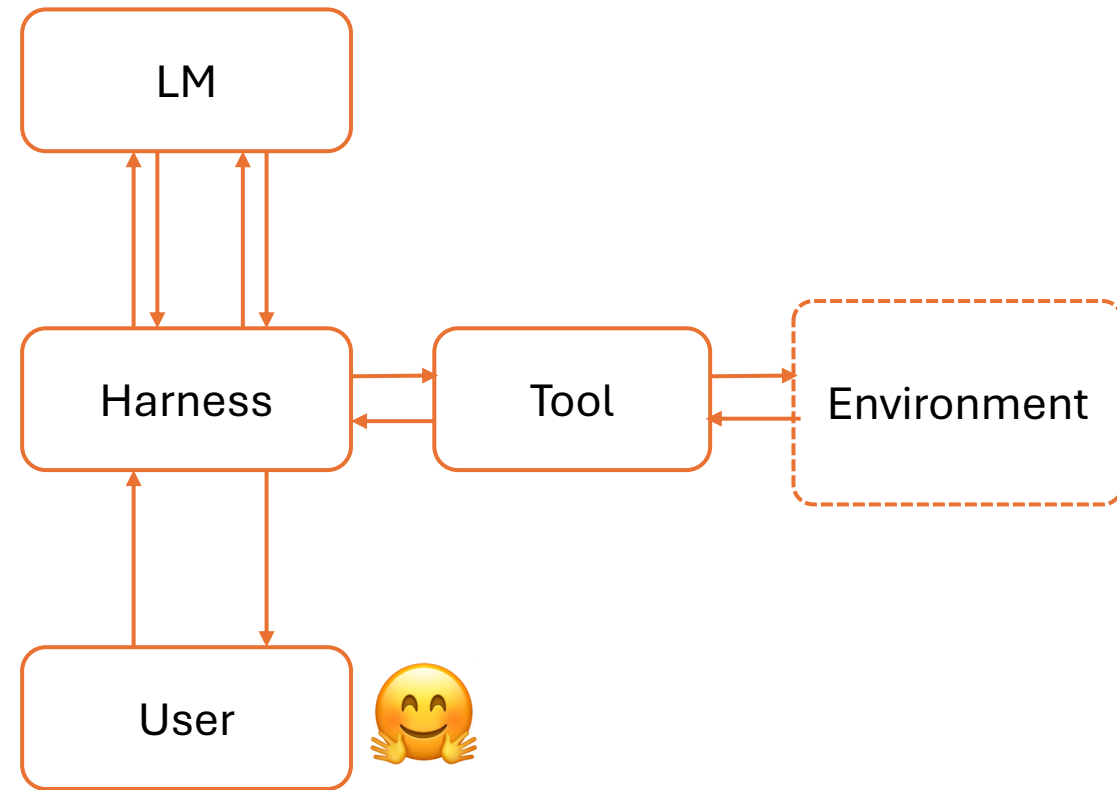
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



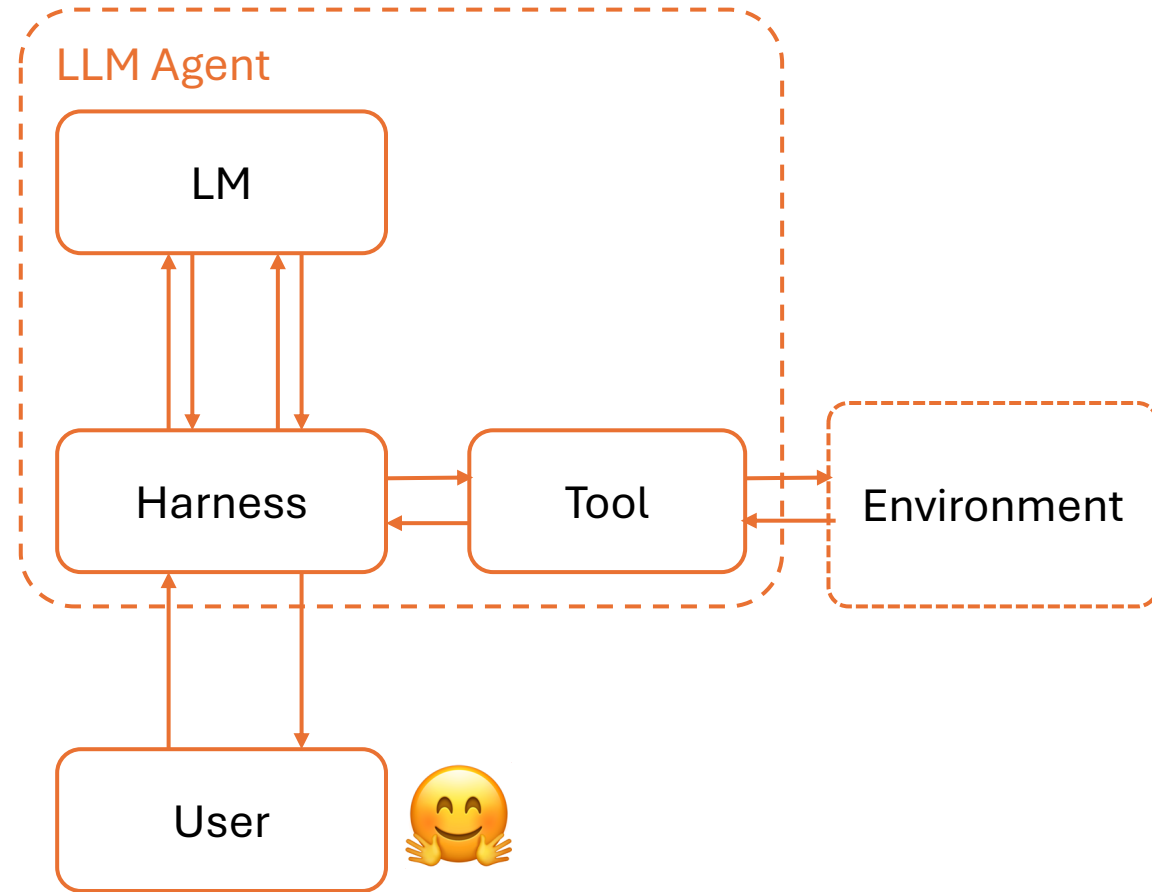
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



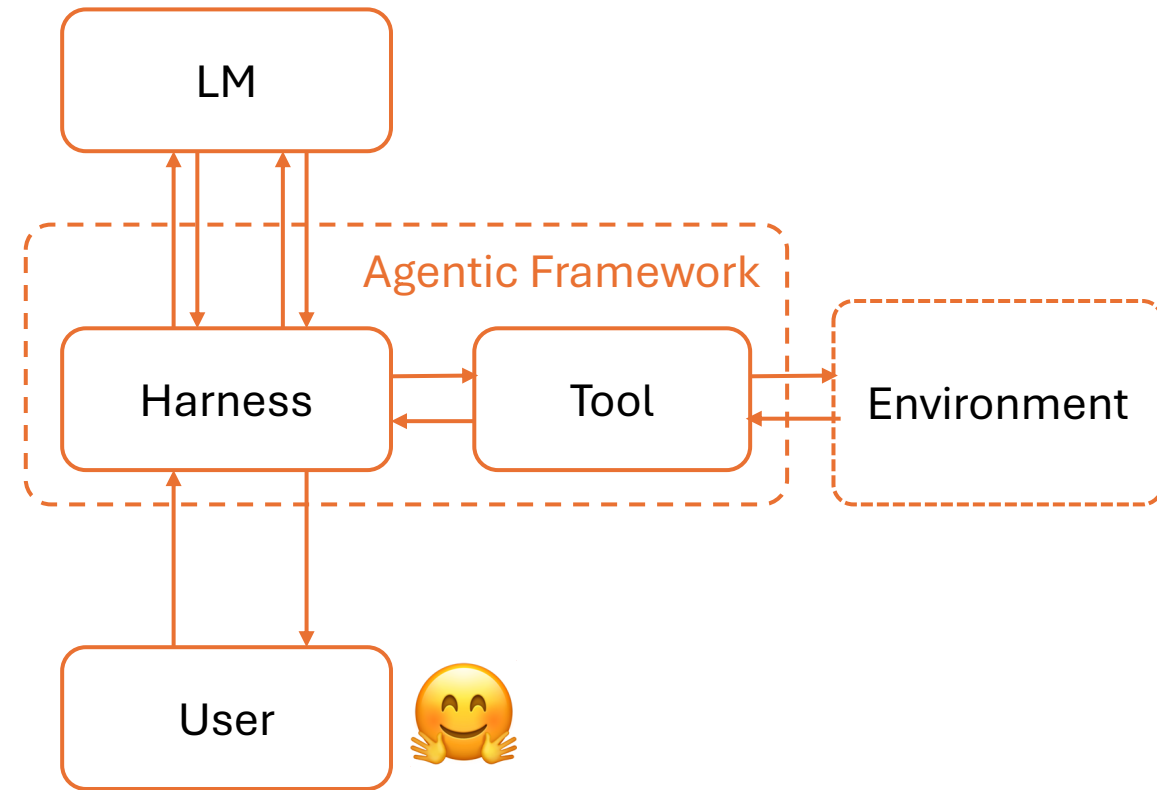
Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



Definitions of Entities

- **User** (human, end-user)
- **Harness** (a computer program)
- **LM** (a neural language model)
- **Environment** (the world)
- **Tool** (access point to the world)



Foundations of Tool Use

Toolformer: Language Models Can Teach Themselves to Use Tools

**Timo Schick Jane Dwivedi-Yu Roberto Dessì[†] Roberta Raileanu
Maria Lomeli Luke Zettlemoyer Nicola Cancedda Thomas Scialom**

Meta AI Research [†]Universitat Pompeu Fabra

Toolformer: Language Models Can Teach Themselves to Use Tools

Timo Schick **Jane Dwivedi-Yu**
Maria Lomeli **Luke Zettlemoyer**
Meta AI Research [†]University of

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Toolformer: Language Models Can Teach Themselves to Use Tools

Timo
Maria

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of the New England Journal of Medicine?")] the Medical Society]

England Journal of Medicine Medical Society]

Out of 1400 participants, [→ 0.29] 29%) passed the

The name derives from "I [MT("tortuga") → turtle]

The Brown Act is California [Act") → The Ralph M. Brown Act] California State Legislature right to attend and participate in legislative bodies.] that require city councils, to hold their

Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

Input: Joe Biden was born in Scranton, Pennsylvania.

Output: Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton, [QA("In which state is Scranton?")] Pennsylvania.

Input: Coca-Cola, or Coke, is a carbonated soft drink manufactured by the Coca-Cola Company.

Output: Coca-Cola, or [QA("What other name is Coca-Cola known by?")] Coke, is a carbonated soft drink manufactured by [QA("Who manufactures Coca-Cola?")] the Coca-Cola Company.

Input: x

Output:

leanu
Scialom

Toolformer: Language Models Can Teach Themselves to Use Tools

Roberto Dessì† Roberta Raileanu

API Name	Example Input	Example Output
Question Answering	Where was the Knights of Columbus founded?	New Haven, Connecticut
Wikipedia Search	Fishing Reel Types	Spin fishing > Spin fishing is distinguished between fly fishing and bait cast fishing by the type of rod and reel used. There are two types of reels used when spin fishing, the open faced reel and the closed faced reel.
Calculator	$27 + 4 * 2$	35
Calendar	ϵ	Today is Monday, January 30, 2023.
Machine Translation	sûreté nucléaire	nuclear safety

Toolformer: Language Models Can Teach Themselves to Use Tools

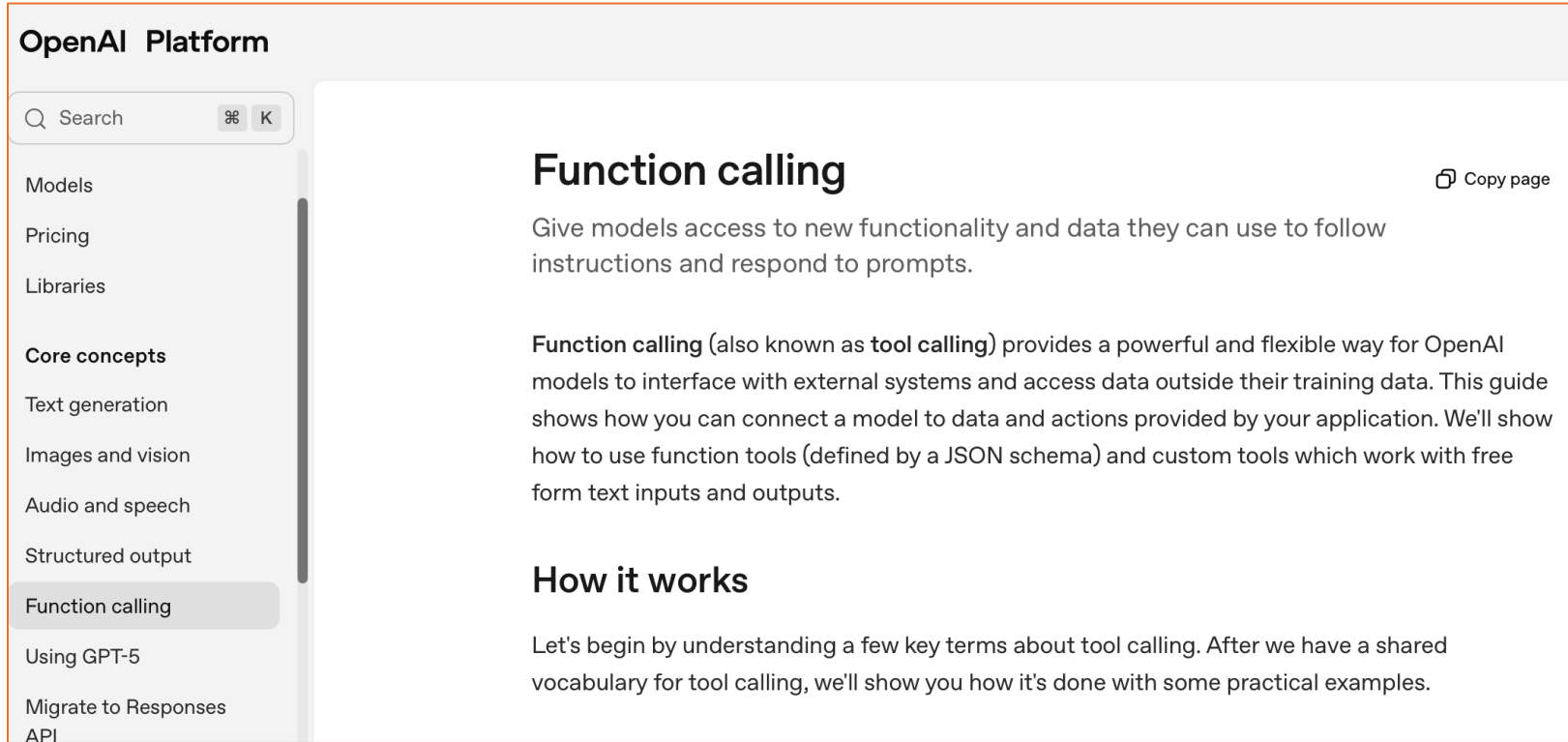
Roberto Dessì† Roberta Raileanu

API Name	Example Input	Example Output
Question Answering	Where was the Knights	New Haven, Connecticut

Model	ASDiv	SVAMP	MAWPS
GPT-J	7.5	5.2	9.9
GPT-J + CC	9.6	5.0	9.3
Toolformer (disabled)	14.8	6.3	15.0
Toolformer	<u>40.4</u>	<u>29.4</u>	<u>44.0</u>
OPT (66B)	6.0	4.9	7.9
GPT-3 (175B)	14.0	10.0	19.8

Table 4: Results for various benchmarks requiring mathematical reasoning. Toolformer makes use of the calculator tool for most examples, clearly outperforming even OPT (66B) and GPT-3 (175B).

Foundations of Tool Use



The screenshot shows the OpenAI Platform documentation interface. On the left is a sidebar with a search bar and a list of navigation items. The main content area on the right features the title 'Function calling', a brief introductory paragraph, a 'Copy page' button, a detailed paragraph explaining the concept, and a section titled 'How it works' with its introductory text.

OpenAI Platform

Search  K

- Models
- Pricing
- Libraries
- Core concepts**
 - Text generation
 - Images and vision
 - Audio and speech
 - Structured output
 - Function calling**
 - Using GPT-5
 - Migrate to Responses API

Function calling

 Copy page

Give models access to new functionality and data they can use to follow instructions and respond to prompts.

Function calling (also known as **tool calling**) provides a powerful and flexible way for OpenAI models to interface with external systems and access data outside their training data. This guide shows how you can connect a model to data and actions provided by your application. We'll show how to use function tools (defined by a JSON schema) and custom tools which work with free form text inputs and outputs.

How it works

Let's begin by understanding a few key terms about tool calling. After we have a shared vocabulary for tool calling, we'll show you how it's done with some practical examples.

Q Search

⌘ K

- Models
- Pricing
- Libraries
- Core concepts
- Text generation
- Images and vision
- Audio and speech
- Structured output
- Function calling
- Using GPT-5
- Migrate to Responses API

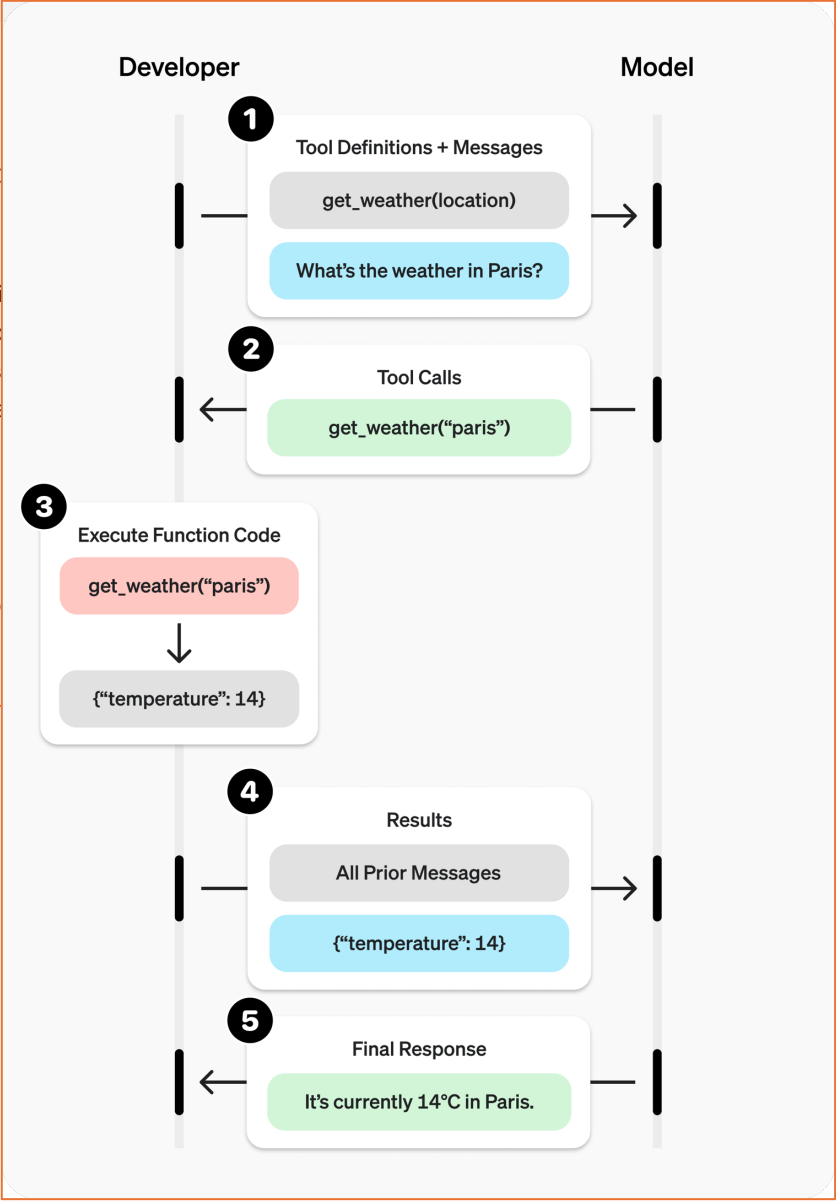
Function calling

Give models access to new functionality and instructions and respond to prompts.

Function calling (also known as **tool calling**) provides models to interface with external systems and access data. This section shows how you can connect a model to data and how to use function tools (defined by a JSON schema) to form text inputs and outputs.

How it works

Let's begin by understanding a few key terms about the vocabulary for tool calling, we'll show you how it's



OpenAI Platform

Search

⌘ K

Models

Pricing

Libraries

Core concepts

Text generation

Images and vision

Audio and speech

Structured output

Function calling

Using GPT-5

Migrate to Responses API

API

Developer

1 Tool Definitions + Messages
get_weather(location)

What's the weather in Paris?

2 Tool Calls
get_weather("paris")

3 Execute Function Code
get_weather("paris")

{ "temperature": 14 }

4 Results
All Prior Messages
{ "temperature": 14 }

5 Final Response
It's currently 14°C in Paris.

Model

```
6 # 1. Define a list of callable tools for the model
7 tools = [
8     {
9         "type": "function",
10        "name": "get_horoscope",
11        "description": "Get today's horoscope for an astrological sign.",
12        "parameters": {
13            "type": "object",
14            "properties": {
15                "sign": {
16                    "type": "string",
17                    "description": "An astrological sign like Taurus or Aquarius."
18                },
19            },
20            "required": ["sign"],
21        },
22    },
23 ]
24
25 def get_horoscope(sign):
26     return f"{sign}: Next Tuesday you will befriend a baby otter."
27
28 # Create a running input list we will add to over time
29 input_list = [
30     {"role": "user", "content": "What is my horoscope? I am an Aquarius."}
31 ]
32
33 # 2. Prompt the model with tools defined
34 response = client.responses.create(
35     model="gpt-5",
36     tools=tools,
37     input=input_list,
38 )
```


OpenAI Platform

Search

Models

Pricing

Libraries

Core concepts

Text generation

Images and vision

Audio and speech

Structured outputs

Function calling

Using GPT-5

Migrate to ReAct

API

Developer

```
6 # 1. Define a list of callable tools for the model
7 tools = [
8     {
9         "type": "function",
10        "name": "get_horoscope",
11        "description": "Get a horoscope for a given date and location.",
12        "parameters": {
13            "type": "object",
14            "properties": {
15                "date": {
16                    "type": "string",
17                    "description": "The date for the horoscope.",
18                },
19                "location": {
20                    "type": "string",
21                    "description": "The location for the horoscope.",
22                },
23            },
24            "required": ["date", "location"],
25        },
26    },
27 ]
28
29 # 2. Create a function to call the model
30 def get_horoscope(date, location):
31     # Create a prompt for the model
32     prompt = f"Get a horoscope for {date} in {location}."
33     response = model.generate_text(prompt, tools=tools, input=input)
34     return response
```

3 Execute Function

get_weather("pa

("temperature":

Copy page

```
43 for item in response.output:
44     if item.type == "function_call":
45         if item.name == "get_horoscope":
46             # 3. Execute the function logic for get_horoscope
47             horoscope = get_horoscope(json.loads(item.arguments))
48
49 # 4. Provide function call results to the model
50 input_list.append({
51     "type": "function_call_output",
52     "call_id": item.call_id,
53     "output": json.dumps({
54         "horoscope": horoscope
55     })
56 })
```

Exemplars of Tool Use

VOYAGER: An Open-Ended Embodied Agent with Large Language Models

**Guanzhi Wang^{1 2}✉, Yuqi Xie³, Yunfan Jiang^{4*}, Ajay Mandlekar^{1*},
Chaowei Xiao^{1 5}, Yuke Zhu^{1 3}, Linxi “Jim” Fan^{1†}✉, Anima Anandkumar^{1 2†}**

¹NVIDIA, ²Caltech, ³UT Austin, ⁴Stanford, ⁵UW Madison

*Equal contribution †Equal advising ✉ Corresponding authors

<https://voyager.minedojo.org>

AIHED00.JO

Building Open-Ended Embodied Agents with Internet-Scale Knowledge

🌟🌟 NeurIPS 2022 Outstanding Paper Award 🌟🌟

Overview

Paper

NeurIPS 2022

Knowledge Base

GitHub

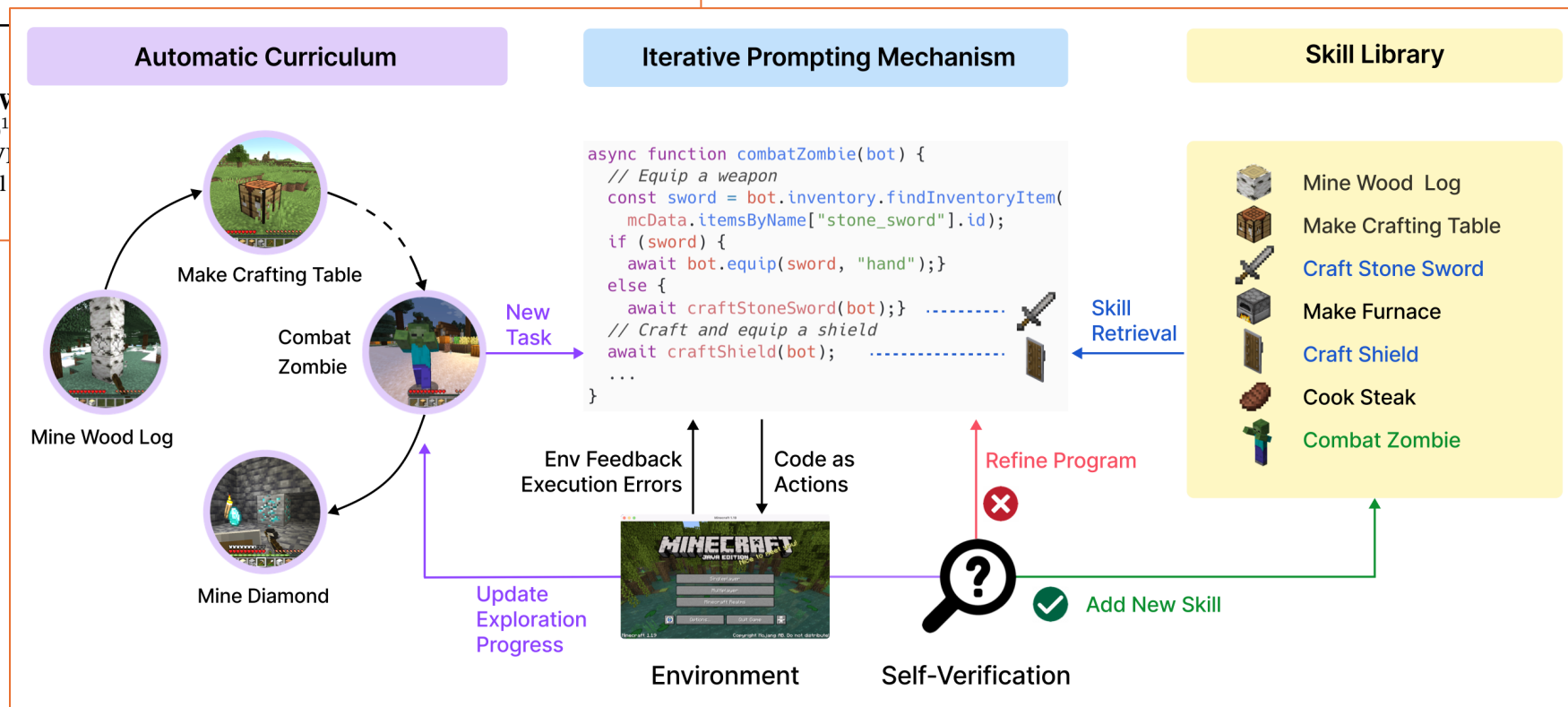
Docs

Team



VOYAGER: An Open-Ended Embodied Agent with Large Language Models

Guanzhi V
Chaowei Xiao¹
¹NVIDIA
*Equal



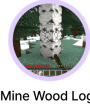
VOYAGER: An Open-Ended Embodied Agent

Automatic Curriculum

Iterative Prompting Mechanism

Skill Library

Chao



Inventory (5/36): {'oak_planks': 3, 'stick': 4, 'crafting_table': 1, 'stone': 3, 'wooden_pickaxe': 1}

GPT-4

Reasoning: Since you have a wooden pickaxe and some stones, it would be beneficial to upgrade your pickaxe to a stone pickaxe for better efficiency.
Task: Craft 1 stone pickaxe.



Biome: river
Inventory (4/36): {'sugar_cane': 3, 'paper': 3, 'fishing_rod': 1, 'wooden_pickaxe': 1}

GPT-4

Reasoning: Since you have a fishing rod in your inventory and you are near a river biome, it would be a good opportunity to catch some fish for food and experience.
Task: Catch 1 fish.



Nearby entities: pig, cat, villager
Health: 12/20
Hunger: 0/20

GPT-4

Reasoning: Your hunger is at 0, which means you need to find food to replenish your hunger. Since there are pigs nearby, you can kill one to obtain raw porkchops.
Task: Kill 1 pig.



Inventory (6/36): {'furnace': 1, 'stone_pickaxe': 1, 'oak_planks': 7, 'crafting_table': 1, 'raw_iron': 4, 'coal': 1}

GPT-4

Reasoning: You have raw iron and coal, and you have a furnace. It's time to smelt the iron to make iron ingots, which can be used to craft better tools and armor.
Task: Smelt 4 raw iron.



Time: night
Nearby entities: zombie
Equipment: 'stone_sword', 'shield'

GPT-4

Reasoning: Since it's night and there's a zombie nearby, it's a good opportunity to try killing the zombie now that you have a stone sword and a shield equipped.
Task: Kill 1 zombie.

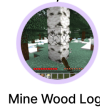
VOYAGER: An Open-Ended Embodied Agent

Automatic Curriculum

Iterative Prompting Mechanism

Skill Library

Chat



Mine Wood Log



Inventory (5/36): {'oak_planks': 3, 'stick': 4, 'crafting_table': 1, 'stone': 3, 'wooden_pickaxe': 1}

GPT-4

Reasoning: Since you have a wooden pickaxe and some stones, it would be beneficial to upgrade your pickaxe to a stone pickaxe for better efficiency.
Task: Craft 1 stone pickaxe.

Environment Feedback

I cannot make stick because I need: 2 more planks
I cannot make stone_shovel because I need: 2 more stick

GPT-4

```
async function craftStoneShovelWithTable(bot) {  
  // If not enough cobblestone, mine cobblestone  
  ...  
  + // If not enough sticks, check if there are  
  + // enough planks in the inventory  
  if (sticksCount < 2) {  
    + const planksCount = bot.inventory.count(  
    +   mcData.itemsByName.acacia_planks.id);  
    + if (planksCount < 2) {  
    +   // Collect or craft planks  
    +   await mineBlock(bot, "acacia_log", 1);  
    +   await craftItem(bot, "acacia_planks", 1);  
    + }  
    + // Craft sticks using planks  
    await craftItem(bot, "stick", 1);  
    await craftItem(bot, "stone_shovel", 1);  
  }  
}
```

Execution Error

throw new Error(`No item named \${name}`);
No item named acacia_axe
at line 18:await craftItem(bot, "acacia_axe", 1);

GPT-4

```
- async function craftAcaciaAxe(bot) {  
+ async function craftWoodenAxe(bot) {  
  // Craft anacia planks and sticks  
  ...  
  
  // Place the crafting table near the bot  
  ...  
  
  - // Craft an acacia axe using 3 acacia planks  
  - // and 2 sticks  
  - await craftItem(bot, "acacia_axe", 1);  
  - bot.chat("Acacia axe crafted.");  
  + // Craft a wooden axe using 3 acacia planks  
  + // and 2 sticks  
  + await craftItem(bot, "wooden_axe", 1);  
  + bot.chat("Wooden axe crafted.");  
}
```

VOYAGER: An Open-Ended Embodied Agent

Automatic Curriculum

Iterative Prompting Mechanism

Skill Library

Chao



Inventory (5/36): ('oak_planks': 3, 'stick': 4, 'crafting_table': 1, 'stone': 3, 'wooden_pickaxe': 1)

GPT-4

Reasoning: Since you have a wooden pickaxe and some stones, it would be beneficial to upgrade your pickaxe to a stone pickaxe for better efficiency.
Task: Craft 1 stone pickaxe.

Environment Feedback

Execution Error

I cannot make stick because I need: 2 more planks
I cannot make stone_s

throw new Error('No item named \${name}');

your inventory and you are near a river
atch some fish for food and experience.

```
async function cr
// If not enou
...
+ // If not enou
+ // enough plan
+ if (sticksCount
+ const planks
+ mcData.item
+ if (planksCount
+ // Collect
+ await mine
+ await craft
+ }
+ // Craft stick
+ await craftItem
+ await craftItem
}
```

(2) Control primitive APIs implemented by us: These APIs serve a dual purpose: they demonstrate the usage of Mineflayer APIs, and they can be directly called by GPT-4.

- `exploreUntil(bot, direction, maxTime = 60, callback)`: Allow the agent to explore in a fixed direction for `maxTime`. The callback is the stopping condition implemented by the agent to determine when to stop exploring;
- `mineBlock(bot, name, count = 1)`: Mine and collect the specified number of blocks within a 32-block distance;
- `craftItem(bot, name, count = 1)`: Craft the item with a crafting table nearby;
- `placeItem(bot, name, position)`: Place the block at the specified position;
- `smeltItem(bot, itemName, fuelName, count = 1)`: Smelt the item with the specified fuel. There must be a furnace nearby;

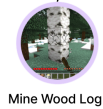
VOYAGER: An Open-Ended Embodied Agent

Automatic Curriculum

Iterative Prompting Mechanism

Skill Library

Chaco



Mine Wood Log



Inventory (5/36): {'oak_planks': 3, 'stick': 4, 'crafting_table': 1, 'stone': 3, 'wooden_pickaxe': 1}

GPT-4

Reasoning: Since you have a wooden pickaxe and some stones, it would be beneficial to upgrade your pickaxe to a stone pickaxe for better efficiency.
Task: Craft 1 stone pickaxe.

Environment Feedback

Execution Error

I cannot make stick because I need: 2 more planks
I cannot make stone_s

throw new Error('No item named \${name}');

your inventory and you are near a river
catch some fish for food and experience.

(2) Control primitive APIs implemented by us: These APIs serve a dual purpose: they demonstrate the usage of Mineflayer APIs, and they can be directly called by GPT-4.

Skill library example 1: craftWoodenPlanks.

```
async function craftWoodenPlanks(bot) {
  const logNames = ["oak_log", "birch_log", "spruce_log", "jungle_log",
    "acacia_log", "dark_oak_log", "mangrove_log"];
  const plankNames = ["oak_planks", "birch_planks", "spruce_planks", "jungle_planks", "acacia_planks", "dark_oak_planks", "mangrove_planks"];
  const logInInventory = logNames.find(logName => bot.inventory.count(mcData.itemsByName[logName].id) > 0);
  if (!logInInventory) {
    bot.chat("No wooden log in inventory. Mining a wooden log...");
    await mineWoodLog(bot);
  }
  const logIndex = logNames.indexOf(logInInventory);
  const plankName = plankNames[logIndex];
  bot.chat(`Crafting 4 ${plankName}...`);
  await craftItem(bot, plankName, 1);
  bot.chat(`4 ${plankName} crafted.`);
}
```


Week 4

- Assignment 1
 - <https://github.com/machine-programming/assignment-1>
 - Due Today!
- Assignment 2
 - Going to be out on Thursday and will due two weeks from now